

AI-Assisted Programming in Software Engineering

A Systematic Literature Review and Socio-Technical Perspective

León Felipe Guevara-Chávez

Received: date / Accepted: date

Abstract Insert your abstract here. Include keywords, PACS and mathematical subject classification numbers as needed.

Keywords AI-assisted programming · Systematic literature review · Socio-technical equilibrium · Human–AI collaboration · Developer productivity · Large Language Models

1 Introduction

Your text comes here. Separate text sections with

2 Methodology

2.1 Research Design

This study follows a Systematic Literature Review (SLR) methodology, structured according to established evidence-based software engineering guidelines [44] and the PRISMA 2020 framework [69] to ensure transparency, rigour, and reproducibility.

The objective is to identify, evaluate, and synthesise empirical evidence regarding the impact of AI-assisted programming tools on software development. The review process was designed as a multi-stage protocol comprising two complementary discovery pathways, screening, eligibility assessment, quality evaluation, epistemic segmentation, and synthesis.

2.2 Research Question

The study is guided by the following research question:

León Felipe Guevara-Chávez
School of Engineering and Science
Tecnológico de Monterrey
E-mail: leon.guevara@tec.mx

- RQ1: What is the impact of AI-assisted programming on software development processes and outcomes?

To address this question, the analysis was structured across seven analytical dimensions: productivity, code quality and technical debt, human–AI interaction, cognitive load, organisational impact, economic impact, and perception and trust.

2.3 Search Strategy

The literature search combined two complementary discovery pathways. The primary pathway (Branch A) used SciSpace, an AI-assisted academic discovery engine that orchestrates retrieval across multiple indexed sources. The complementary pathway (Branch B) used author-executed direct queries on three academic databases: Scopus, ProQuest, and arXiv. Both pathways targeted publications between 2021 and 2026, reflecting the emergence and rapid evolution of modern AI-assisted coding tools. Bibliographic information for individual papers was occasionally imported from publisher sites (IEEE Xplore, ACM Digital Library, SpringerLink, ACL Anthology); these channels served solely as bibliographic-import mechanisms and are not counted as search sources, since the discovery of each such paper had already occurred on one of the four search sources above.

2.3.1 Branch A: SciSpace AI-assisted retrieval

A structured search instruction was provided to SciSpace specifying the review topic, research questions, the time window, inclusion and exclusion criteria, required output fields, and quality-control constraints. SciSpace executed the instruction across three internal retrieval modes: (i) SciSpace Deep Search (855 records from two semantic queries: 371 and 484), (ii) SciSpace Full-Text Search (200 records from two queries of 100 each), and (iii) Google Scholar, queried programmatically by SciSpace as part of its retrieval pipeline (200 records from two queries of 100 each). The combined identification yielded 1,255 records, reduced to 863 unique records after SciSpace’s internal deduplication. These records were then carried through SciSpace’s two-stage screening pipeline (title/abstract scoring with a cutoff of 4.0, followed by full-text scoring with a cutoff of 4.5), with each AI-assisted decision validated manually by the author against the eligibility criteria described in Section 2.4. This branch retained 355 records at title/abstract screening and 349 records at full-text screening. The complete archive of SciSpace’s search outputs (query logs, retrieved record CSVs, and screening artefacts) is preserved as supplementary material.

2.3.2 Branch B: Author-executed database queries

The author-executed database pathway returned 1,852 records across three sources: 1,713 from Scopus (two exports, dated 5 April and 3 May 2026), 89 from ProQuest, and 50 records browsed on arXiv through a structured keyword search combining AI-assisted-programming, software-engineering, and outcome-related terms (arXiv displays 50 records per result page; the search yielded a single page of results relevant to the review scope). After title/abstract screening (113 retained) and

full-text assessment (107 retained), cross-branch deduplication against the SciSpace branch removed 32 duplicate records, yielding a net contribution of 69 unique records. Search strings, filters, and export timestamps for each query are preserved in the supplementary materials of the project.

2.4 Use of Artificial Intelligence in the Review Process

Artificial intelligence tools were used to support specific tasks in the review process. SciSpace functioned as an AI-assisted discovery engine, orchestrating queries across its proprietary semantic indexes, full-text search, and Google Scholar, and as a first-pass relevance scorer for title/abstract and full-text screening. Decisions emitted by SciSpace were treated strictly as assistive recommendations: every inclusion, exclusion, classification, and quality-assessment decision was validated and recorded manually by the author. In addition, the corpus includes a small number of studies published in Chinese, Russian, and German; for these, AI-assisted translation (ChatGPT) was used to support the author's reading and eligibility assessment, with the same inclusion criteria applied as for studies in English, Spanish, and Portuguese, which the author read directly. AI tools were not used to perform synthesis, data extraction, or interpretation of findings.

2.5 Inclusion and Exclusion Criteria

Eligibility criteria were applied identically to both branches and at every screening stage. Studies were included if they reported empirical evaluation (quantitative, qualitative, or mixed-methods) of AI-assisted programming or code generation; evaluated modern AI tools (e.g., Copilot, ChatGPT, Codex, Gemini, Claude Code, CodeWhisperer, Cursor, or comparable systems); reported results allowing analytical interpretation; and were published between 2021 and 2026.

Studies were excluded if they were opinion papers without empirical content, focused on unrelated NLP tasks, were duplicates of records already accepted from another source, or fell outside the topical scope of AI-assisted software development. Surveys, systematic literature reviews, and conceptually adjacent works were reclassified to the Related Work or Support set rather than discarded, in order to preserve their contribution to the framing of the synthesis.

2.6 Operational Taxonomy of Records

To maintain a transparent and auditable separation between evidence used for synthesis and evidence used for framing, each retained record was assigned to one of five operational categories, summarised in Table 1.

The Corpus comprises the primary empirical evidence base: studies with empirical results directly analysed across the seven analytical dimensions, weighted by an epistemic segmentation described below. The Related Work or Support category comprises surveys, systematic reviews, and conceptually adjacent works providing theoretical and contextual grounding; these are cited in the Introduction and Discussion but are not counted in corpus statistics. The Duplicated category

Table 1 Operational taxonomy of records

Category	Count (<i>n</i>)
Corpus (primary empirical evidence base)	312
Related Work or Support	67
Duplicated	21
No Full Text	22
Excluded	5

preserves preprint/published-version pairs of records already represented in another sheet (internal SciSpace duplicates), retained for audit transparency. The No Full Text category records studies whose full text could not be retrieved, excluded from empirical analysis but retained for traceability. The Excluded category comprises records that entered the structured screening pipeline but failed quality or scope criteria at full-text assessment.

Within the Corpus, a further epistemic segmentation governs interpretive weight: CORE (168 studies), SUPPORTING (120 studies), PERIPHERAL (18 studies), and REMOVE_CANDIDATE (6 studies). All descriptive statistics reported in this paper are computed over the full Corpus ($n = 312$), while strong claims are preferentially anchored in CORE studies and triangulated with SUPPORTING evidence.

2.7 Study Selection Process and PRISMA Flow

The selection process followed the four-phase PRISMA 2020 framework (Identification, Screening, Eligibility, Inclusion) and was applied to both branches in parallel, as shown in Fig. 1.

At identification, the SciSpace pipeline (Branch A) returned 1,255 records, reduced to 863 unique records after its internal deduplication; the author-executed database pathway (Branch B) returned 1,852 records across Scopus, ProQuest, and arXiv. At title/abstract screening, 355 records were retained from Branch A and 113 from Branch B. At full-text screening, 349 records were retained from Branch A and 107 from Branch B. Cross-branch deduplication then removed 32 Branch B records that duplicated SciSpace hits, yielding a net Branch B contribution of 69 records.

The two branches thus converged into an integrated dataset of 418 unique records from structured search (349 from Branch A and 69 net from Branch B). This dataset was subjected to manual quality refinement and dimensional classification, and distributed across the five operational sheets defined in Section 2.5, yielding 308 records for the Corpus and 62 for Related Work or Support (together with 21 Duplicated, 22 No Full Text, and 5 Excluded records, totalling 418).

A supplementary discovery procedure was then applied selectively to address residual gaps and to preserve methodological transparency. This procedure covered three situations: (i) targeted keyword search prompted by gap diagnosis when a specific analytical dimension showed insufficient empirical depth; (ii) citation chasing from references in already-retrieved papers; and (iii) opportunistic retrieval during full-text resolution, including cases where related-link panels or alternative result lists surfaced conceptually relevant works. All candidates identified through

this procedure were subjected to the same eligibility criteria and quality-assessment framework as papers retrieved through the two structured pathways.

This supplementary procedure added four primary studies to the Corpus (4 of 312, or 1.3%) and five additional studies to Related Work or Support (5 of 67, or 7.5%), bringing the final totals to 312 and 67 respectively. One foundational study published in 2020 (Brachten et al., on cognitive-load reduction by virtual agents) was retained explicitly as a Related Work reference, providing the conceptual basis for the cognitive-load–performance relationship subsequently extended by the field toward AI-assisted coding tools; it is not counted in the empirical corpus statistics. The full audit trail for the nine supplementary records, including provenance and discovery type, is provided in the supplementary materials.

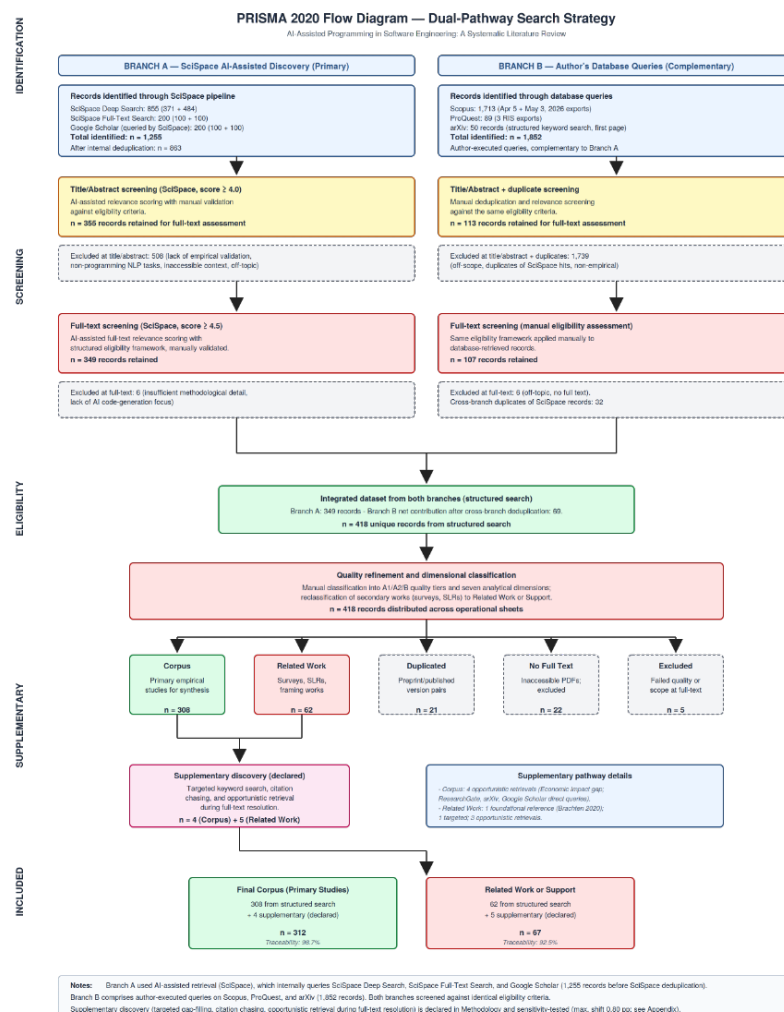


Fig. 1 PRISMA 2020 dual-pathway flow diagram for the systematic literature review.

2.8 Quality Assessment

A multi-criteria evaluation framework was used to assess study quality, considering empirical rigour (e.g., experimental design, sample size), methodological strength (e.g., causal inference versus observational design), venue quality, completeness and transparency of reporting, and relevance to the research question. This framework was applied consistently across all studies to ensure comparability and reduce subjective bias, and was conducted manually to ensure careful evaluation of study design and reporting quality.

Studies were classified into three categories: A1 (high rigour, e.g., controlled experiments or large-scale empirical or causal studies), A2 (moderate rigour, e.g., user studies, case studies, or structured evaluations), and B (supporting evidence, e.g., perception studies, qualitative analyses, or model-based approaches). Of the 312 studies in the Corpus, 146 (46.8%) were classified as A1, 132 (42.3%) as A2, and 34 (10.9%) as B. This classification was used to prioritise higher-quality studies (A1 and A2) in the synthesis and interpretation of results, while category B studies were primarily used to provide contextual support and complementary insights.

2.9 Data Extraction and Synthesis

Data extraction was performed by systematically reviewing each selected study and recording key attributes using a structured extraction template to ensure consistency across studies. For each study, the following data were extracted: study type and methodological classification, tools evaluated, evaluation metrics, key findings, and associated analytical dimensions.

Given the heterogeneity of study designs, metrics, and evaluation contexts, a formal meta-analysis was not feasible. Instead, a structured synthesis approach was adopted to identify consistent patterns and trends across studies, combining quantitative aggregation (e.g., frequency distributions) with qualitative synthesis to identify patterns, contradictions, and research gaps across dimensions. This approach is consistent with established practices in systematic literature reviews, where high variability limits the applicability of statistical meta-analysis.

2.10 Reproducibility and Sensitivity

All search exports and the integrated relation of studies are preserved as supplementary materials, including SciSpace’s complete pipeline artefacts (semantic-search CSVs, full-text-search CSVs, internally executed Google Scholar query CSVs, abstract- and full-text-screening CSVs, and SciSpace’s own PRISMA documentation). Of the 312 primary studies in the final Corpus, 308 (98.7%) are directly traceable to documented structured-search outputs; of the 67 papers in Related Work or Support, 62 (92.5%) are similarly traceable; and all 5 papers in the Excluded set are traceable to structured queries.

To assess the robustness of the reported figures to the inclusion of the nine supplementary-discovery records, a sensitivity analysis was conducted by recomputing all principal distributions (primary dimension, quality classification, epistemic segmentation, and publication year) for the full Corpus ($n = 312$) and for the

Corpus with the four supplementary studies removed ($n = 308$). Across all four axes, no reported percentage shifts by more than 0.80 percentage points, with the largest single shift occurring in the 2025 publication-year share. This confirms that the inclusion of supplementary studies does not materially alter the empirical profile of the corpus, and that the principal findings hold whether or not these studies are included.

3 Results

3.1 Corpus Profile and Evidence Hierarchy

The final corpus comprises 312 studies, reflecting a heterogeneous body of evidence in terms of methodological rigour, evaluation context, and analytical focus. Studies were classified into three quality categories: A1 (high rigour), A2 (moderate rigour), and B (supporting or qualitative evidence).

Of the 312 studies, 146 (46.8%) were classified as A1, 132 (42.3%) as A2, and 34 (10.9%) as B, as shown in Fig. 2. This distribution indicates that the majority of the evidence base is grounded in empirical studies with moderate-to-high methodological rigour.

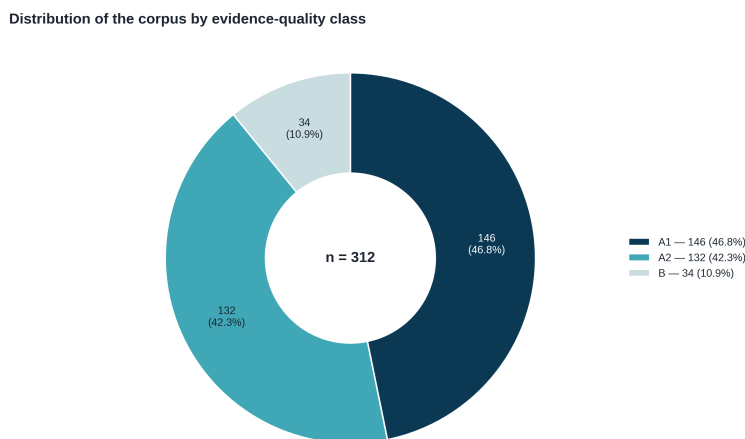


Fig. 2 Distribution of the final corpus by evidence class (A1, A2, and B).

In terms of analytical dimensions, as shown in Fig. 3, the corpus is markedly concentrated in two areas: code quality and technical debt accounts for 45.2% of studies ($n=141$), and human–AI interaction for a further 36.9% ($n=115$). Together, these two dimensions represent more than four-fifths of the corpus. The remaining studies are distributed across productivity (8.3%, $n=26$), perception and trust (4.2%, $n=13$), cognitive load and human factors (2.9%, $n=9$), organisational impact (2.2%, $n=7$), and economic impact (0.3%, $n=1$).

This concentration represents a substantial reorientation of the evidence base relative to earlier framings of the literature, which had often emphasised produc-

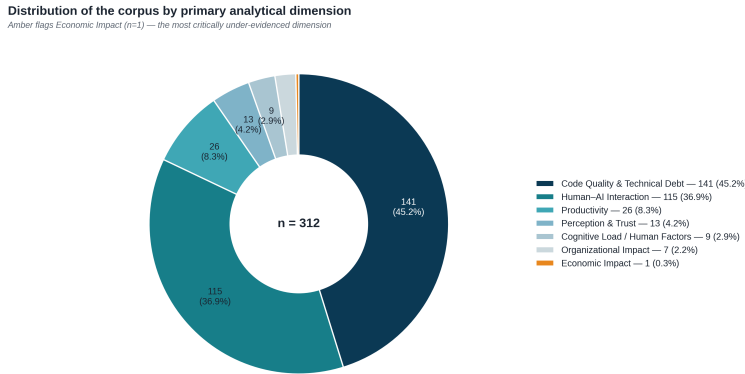


Fig. 3 Distribution of the corpus across the seven analytical dimensions (primary dimension classification). Amber flags Economic Impact (n=1), the most critically under-evidenced dimension.

tivity as the dominant research focus. In the present corpus, productivity-focused studies form a comparatively small subset, while the bulk of empirical attention is directed toward the reliability, security, and maintainability of AI-generated code (code quality and technical debt) and toward the iterative, prompt-driven dynamics of developer–AI collaboration (human–AI interaction). Economic impact, organisational impact, and cognitive load remain markedly underexplored, each represented by fewer than ten studies, with economic impact represented by a single study in the corpus.

Quality classification varies systematically across dimensions, as shown in Fig. 4. Code quality and technical debt exhibits the highest concentration of high-rigour evidence, with 55.3% of its studies classified as A1, reflecting the prevalence of large-scale empirical analyses, controlled benchmarking, and security-focused evaluations in this area. Human–AI interaction shows a more balanced split between A1 and A2 evidence (45.2% and 44.3%, respectively), consistent with a mix of large-scale interaction-log studies and smaller controlled or observational user studies. The remaining dimensions are dominated by A2 evidence, with organisational impact showing the highest proportion of category-B studies (42.9%), reflecting its reliance on qualitative and case-study evidence. Economic impact, the smallest dimension in the corpus, is composed entirely of category-B evidence (100%), underscoring it as the dimension most in need of methodologically stronger studies.

Beyond the primary-dimension classification, an epistemic segmentation was applied to capture the interpretive weight of each study within the synthesis, independently of its analytical dimension. Studies were segmented into CORE (n=168, 53.8%), SUPPORTING (n=120, 38.5%), PERIPHERAL (n=18, 5.8%), and REMOVE_CANDIDATE (n=6, 1.9%). CORE studies anchor the principal claims advanced in each dimension; SUPPORTING studies provide corroborating or contextual evidence; PERIPHERAL studies offer tangential but relevant findings; and REMOVE_CANDIDATE studies, while retained in the corpus for transparency, contribute marginally to the synthesis and are flagged for potential exclusion in future refinements. This segmentation is broadly consistent across dimensions, with code quality and technical debt (77 CORE of 141) and human–AI interaction (60

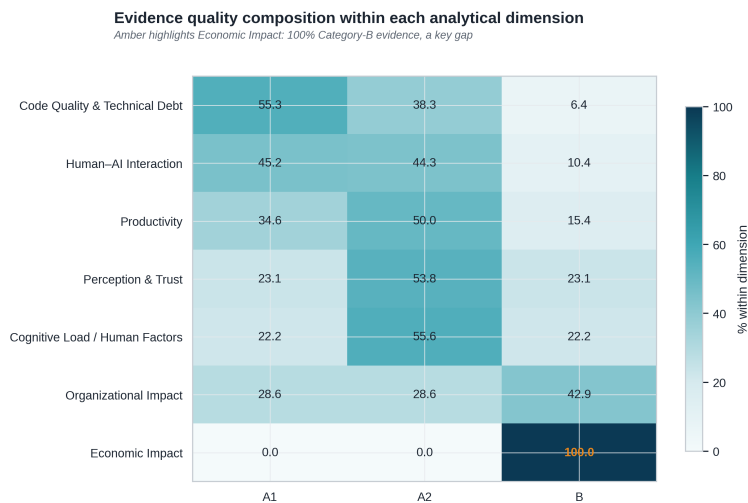


Fig. 4 Evidence quality composition within each analytical dimension. Amber highlights Economic Impact: 100% category-B evidence, a key gap.

CORE of 115) each anchoring more than half of their respective dimensions in CORE evidence.

Finally, the corpus spans publications from 2022 to 2026, with a pronounced concentration in the most recent years: 2024 (n=120, 38.5%) and 2025 (n=121, 38.8%) together account for more than three-quarters of the corpus, while 2026 already contributes 31 studies (9.9%) at the time of the search cutoff. Earlier years are represented more sparsely, with 34 studies (10.9%) from 2023 and 6 studies (1.9%) from 2022. This temporal distribution reflects the rapid and recent expansion of empirical research on AI-assisted programming, consistent with the accelerating adoption of LLM-based coding tools over the period under review. Fig. 5 shows how this growth is distributed across dimensions: code quality and technical debt and human-AI interaction account for the bulk of the increase from 2023 onward, while the apparent decline in 2026 reflects the partial-year search cutoff rather than a genuine slowdown in research activity.

Beyond volume, the methodological rigour of the corpus has also shifted over time. As shown in Fig. 6, the share of A1 (high-rigour) studies rose from 16.7% in 2022 to 54.7% in 2026, indicating that the field is maturing methodologically alongside its rapid growth in volume. This trend is examined further in the Discussion.

This classification enables a weighted synthesis, where conclusions are primarily grounded in stronger empirical evidence and in CORE studies, with SUPPORTING and PERIPHERAL evidence used for triangulation and contextualisation. It also reveals a clear concentration of research efforts on the technical reliability of AI-generated code and on the dynamics of human-AI interaction, with comparatively limited attention to organisational, economic, and cognitive dimensions.

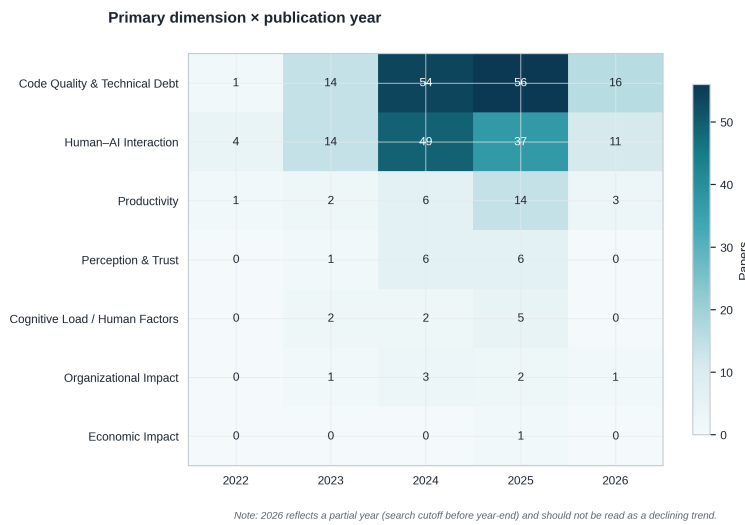


Fig. 5 Primary dimension by publication year. 2026 reflects a partial year (search cutoff before year-end) and should not be read as a declining trend.

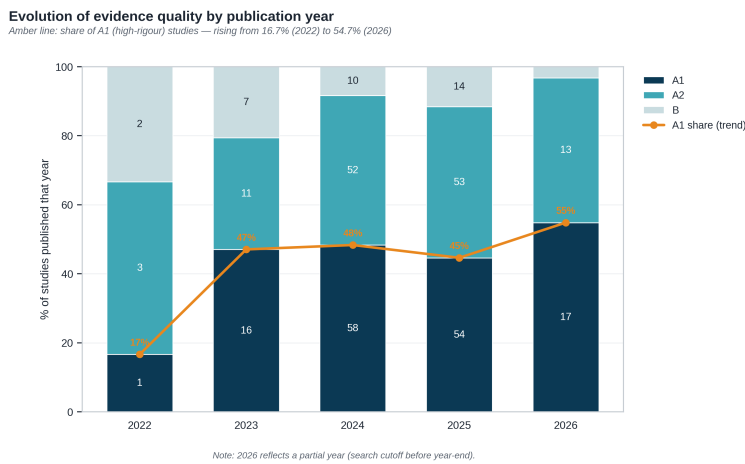


Fig. 6 Evolution of evidence quality by publication year. The amber line tracks the share of A1 (high-rigour) studies.

3.2 Code Quality and Technical Debt

Code quality and technical debt is the largest and most extensively developed dimension in the corpus, accounting for 45.2% of studies ($n=141$; Section 3.1). Evidence in this dimension converges on a structural pattern: AI-assisted programming reliably produces code that satisfies narrow, benchmark-style correctness criteria, but this performance does not transfer cleanly to the conditions under which software is actually maintained, secured, and integrated. The following sub-sections trace this pattern across benchmark performance, real-world generalisation, defect

and structural error rates, security vulnerabilities, maintainability and dependency management, multi-turn and agentic effects, and human-versus-AI comparisons.

3.2.1 Benchmark Performance and the Realism Gap

Synthetic benchmarks remain the most common instrument for evaluating AI-generated code, and the corpus confirms that strong benchmark performance is achievable: Claude Sonnet 4 reached a 95.1% success rate on the 164-problem HumanEval Python benchmark, with comparable assistant evaluations reporting success rates around 40% under more varied task-difficulty conditions. However, this performance does not reliably predict repository-level code quality or maintenance outcomes, a pattern supported by seven studies spanning benchmark evaluation, repository mining, patch analysis, and agentic evaluation [37, 8, 19, 3, 1, 115, 24]. When stricter, EvalPlus-style hidden-test regimes are applied to the same benchmarks, apparent pass@k performance drops by as much as 28.9%, indicating that headline benchmark figures substantially overstate functional reliability under more rigorous testing [58].

This realism gap widens further at repository scale. Agentic systems evaluated on real-world GitHub issues, such as SWE-Bench-style benchmarks, report repair rates around 31–33%, and stricter data-quality filtering on the same benchmark family can lower resolution rates to below 4% [3, 19]. A large-scale repository-mining study of 304,362 verified AI-authored commits similarly finds that benchmark-level success coexists with persistent technical debt once code is merged into production codebases [59]. Taken together, these results indicate that benchmark pass rates and repository-level reliability are only weakly coupled, and that claims of AI coding competence drawn solely from synthetic benchmarks should be treated as bounded rather than generalisable.

3.2.2 Defect Rates and Structural Error Patterns

Beyond benchmark-specific failure, the corpus documents substantial and recurring defect rates in AI-generated code under realistic evaluation conditions. One large-scale quality analysis of ChatGPT-generated Python code found that 76.46% of generated snippets and 84.51% of modified snippets contained at least one quality issue [78]. Studies focused specifically on bug characterisation report that a meaningful share of generated solutions fail to produce functional code even after revision, with one defect-characterisation study finding 19.51% of cases unable to generate a working solution [26], and a code-translation study attributing 33.5% of translation bugs to data-handling errors rather than syntactic mistakes [71]. Regression-testing evidence adds a further layer: fine-tuning or adapting models across programming languages can introduce syntax regressions of up to 12%, even when the underlying generation capability appears to improve [1].

These structural error patterns are corroborated by the broader consensus evidence in the corpus, which identifies persistent technical debt after merging as a strong, repeatedly observed pattern across repository mining, pull-request analysis, and patch-safety studies [59, 12, 91, 20, 83, 30, 86]. Critically, code that passes immediate functional checks can still carry latent maintenance or security liabilities that surface only once it is exercised under real usage conditions, indicating

that defect rates measured at generation time systematically understate downstream risk.

3.2.3 Security Vulnerability Exposure

Security vulnerability is the most thoroughly evidenced sub-theme in the corpus, with ten studies converging on a strong consensus claim: security vulnerabilities are systematically introduced into AI-generated code, but the precise risk profile depends on programming language, CWE family, task type, and prompting context [73, 92, 86, 5, 4, 30, 25, 100, 80, 6]. Reported vulnerability rates vary considerably with evaluation design: foundational work on GitHub Copilot found that approximately 40% of generated programs in security-relevant scenarios were vulnerable [73], while large-sample evaluations such as SALLMS assessed security properties across 17,000 generated samples [92], and comparative human-versus-AI studies report vulnerability rates around 33–49% depending on the comparison baseline [5].

Security repair is consistently weaker and less reliable than security diagnosis, a moderate-consensus pattern supported across eight studies [4, 25, 10, 83, 100, 80, 73, 30]. Repair-oriented evaluations report partial success at best, with patch-level repair outcomes in the range of 16–27% depending on vulnerability type and repair strategy [4]. At the same time, prompt-engineering interventions show that mitigation is genuinely possible: targeted prompt design reduced insecure generated samples by up to 16% and increased the proportion of secure code by up to 85% in one study [80]. This indicates that security risk in AI-generated code is conditional and addressable through deliberate intervention, rather than an immutable property of the underlying models, but that such mitigation requires active human effort rather than occurring by default.

3.2.4 Maintainability, Complexity, and Dependency Management

Maintainability evidence in the corpus is mixed in a specific and informative way: AI-generated code can be concise or superficially readable in narrow tasks while simultaneously accumulating code smells, duplication, structural complexity, or downstream maintenance burden, a strong-consensus pattern supported by six studies using metric-based, pull-request-based, and downstream-tracking designs [37, 78, 12, 19, 24, 31]. In some constrained tasks, AI-generated solutions used markedly fewer lines of code than human-written equivalents, with reductions of up to 60–86% in selected benchmark tasks [37]; however, this brevity does not consistently correspond to lower long-term maintenance cost. A large-scale empirical study of AI-generated code in production repositories found that 89.1% of identified issues were classified as code smells, with 24.2% of affected code surviving long-term and 41.1% of security-relevant flaws persisting in the codebase after merge [59].

Dependency, import, and reproducibility failures emerge as a distinct and systematic sub-theme rather than a minor extension of functional correctness, supported by a moderate consensus across eight studies [104, 102, 51, 115, 19, 99, 59]. An empirical study of dependency gaps in LLM-based coding agents reported dependency-related failure figures as high as 89.2%, alongside a 44.0% rate of incomplete or misspecified listed dependencies [104]. Library hallucination, where generated code references packages or APIs that do not exist or do not match the specified version, is documented as a recurring and separately diagnosable

failure mode [102,51], reinforcing the case that reproducibility and dependency management constitute a CQTD concern in their own right rather than a symptom of generic incorrectness.

3.2.5 Multi-Turn Interaction and Agentic Effects

A further moderate-consensus pattern, supported by eight studies, establishes that multi-turn interaction and agentic workflows do not monotonically improve code quality: depending on trajectory, additional turns or additional tool autonomy can repair, preserve, or amplify defects [19,104,47,116,83,1,36,3]. An analysis of developer-LLM conversations found that import-related errors decreased from 48.3% to 44.6% across successive conversational turns, indicating genuine but incomplete improvement through iteration [116]. Agentic patch evaluation across 4,892 patches from ten different coding agents revealed divergent file- and function-level modification patterns relative to gold-standard patches, showing that increased autonomy expands the surface area over which quality and security risk can manifest [19]. This expansion of risk surface with autonomy is itself a moderate-consensus finding, supported by evidence that insecure agentic behaviours occur at rates including 21% and 56.61% depending on the study and task setting [47]. A benchmark that reconstructs the exact repository state and natural-language requirement under which a human developer originally introduced a real-world vulnerability extends this picture with more fine-grained, agent-specific evidence: across three popular code agents and five backbone LLMs, the best-performing combination produced output that was both functionally correct and free of the reconstructed vulnerability in only 23.8% of cases, and an agent’s likelihood of introducing a given vulnerability type was largely unrelated to how frequently that type occurred in the underlying dataset [16]. The same benchmark further shows that command-line agentic interfaces, despite achieving the highest functional correctness among all evaluated configurations, did not achieve correspondingly high security rates, indicating that scaffolds optimised for completing a task can systematically under-prioritise the security of the result.

These dynamics connect directly to verification and modification effort, which the corpus shows only partially offsets generation-speed gains rather than eliminating the need for human oversight [37,98,116,91,13,1,99]. In one controlled comparison, incorrect but minimally modifiable code occurred in 8.7% of generated outputs, with time savings from correcting near-correct code ranging from 8.9% to 71.3% relative to writing code from scratch [37]. Eye-tracking and IDE-action studies of developer validation behaviour confirm that this verification work is cognitively substantive rather than perfunctory, requiring developers to actively trace logic and confirm correctness rather than passively accepting AI-generated suggestions [98].

3.2.6 Human-versus-AI Quality Comparisons

Direct comparisons between human-written and AI-generated code are consistently context-dependent rather than categorical, a strong-consensus pattern supported by eight studies: AI systems can outperform human baselines on some surface metrics while underperforming on correctness, security, or maintainability in other settings [37,8,20,5,78,91,12,73]. In one comparative study, GitHub Copilot solved

50.0% of a curated task set while the broader evaluated AI tool set solved only 20.6% of tasks overall, illustrating substantial variation even among AI systems evaluated under the same conditions [37]. At the same time, the strongest current models achieve benchmark scores that meet or exceed typical human baselines on narrow, well-specified tasks, with top-performing models reaching 95.1% and 94.5% on HumanEval in one comparative analysis [8].

The direction of this comparison reverses, however, once security and downstream maintainability are taken into account: studies comparing Copilot’s behaviour regarding vulnerability introduction with that of human developers, and studies tracking downstream maintainability effects, both indicate that AI-generated code does not categorically outperform human-written code once these dimensions are considered [5, 12]. This reversal is the empirical basis for treating correctness, security, maintainability, reproducibility, and human validation as analytically separate constructs within CQTD rather than collapsing them into a single quality score, a strong-consensus conclusion supported by seven studies spanning repository-scale, downstream, reproducibility, validation, and regression-testing designs [59, 86, 12, 104, 98, 91, 1].

3.2.7 Synthesis

Across all seven sub-themes, the CQTD evidence base converges on a single structural conclusion: generation efficiency in AI-assisted programming is empirically real, but it is systematically and substantially offset by verification overhead. Strong benchmark performance, concise output, and partial security mitigation are all genuine and well-documented capabilities; equally well documented are the realism gap between benchmark and repository-level performance, the persistence of technical debt and security vulnerabilities after merge, the non-monotonic effect of additional interaction turns and agentic autonomy, and the necessity of treating correctness, security, maintainability, and reproducibility as distinct rather than interchangeable quality constructs. This pattern provides the most methodologically robust empirical grounding in the corpus for the verification-overhead component of the socio-technical equilibrium framework developed in this study, a connection examined further in the Discussion.

3.3 Human–AI Interaction

Human–AI interaction is the second-largest dimension in the corpus, accounting for 36.9% of studies ($n=115$; Section 3.1). The evidence base for this dimension is organised around seven sub-themes: prompting dynamics and conversational strategies, workflow integration and collaboration patterns, trust and reliance behaviour, developer experience and perceived value, learning and educational contexts, robustness and agentic reliability, and identity and socio-professional continuity. Across these sub-themes, AI-assisted programming appears less as a replacement of developer expertise than as a reconfiguration of interaction, verification, judgement, and responsibility: developers do not simply receive code, they negotiate it.

3.3.1 Prompting Dynamics and Conversational Strategies

Prompting functions as the interaction infrastructure of AI-assisted programming rather than as a generic natural-language skill. Developers express intent, surface local context, specify constraints, and negotiate the form of repair, explanation, or continuation needed to make a generated artefact usable [67, 7, 43]. This positions prompting as situated software work: professional use involves assembling task goals, surrounding code, desired edit behaviour, and follow-up constraints into an interaction that progressively shapes model output. This interaction channel is technically consequential rather than cosmetic. Ambiguous, contradictory, and incomplete task descriptions degrade code-model robustness [50]; semantically preserving natural-language perturbations can still alter results [17]; and faulty premises can mislead generation unless critical checking is incorporated into the interaction [53]. Quantitative robustness evidence reinforces this point directly: only 27.43% of GitHub Copilot paraphrase cases preserved the same correct output across semantically equivalent prompt variants [61], and both superficial programming-problem transformations and simulated Copilot-style completion context measurably alter reliability [90, 39]. Prompting can also span the full development lifecycle rather than isolated code requests, proceeding through cycles of generation, inspection, correction, and continuation from requirements through documentation [52, 28, 107].

3.3.2 Workflow Integration and Collaboration Patterns

AI assistance shifts developer work rather than removing it. The central workflow change is a redistribution of labour towards reading, editing, rejection, validation, orchestration, and trace management [64, 94, 109]. One influential cost model frames AI-assisted programming explicitly in these terms: the value of a generated suggestion depends on the time and effort required to read, understand, and repair it, not on how quickly text appears [64]. This redistribution is visible both in everyday solo and pair programming, where Copilot and ChatGPT change how developers move between assistance, oversight, interruption, and collaborative sense-making [94], and in automated program repair, where localisation, patch generation, and validation must be explicitly orchestrated for the repair process to function at all [109]. Collaborative-trace evidence extends this workflow claim beyond the editor: shared ChatGPT conversations appeared in 33.2% of pull-request conversations and 36.9% of issue conversations in one large-scale GitHub corpus [34], with related evidence on AI-generated code and self-admitted generative-AI use confirming that this delegation-with-validation pattern is already embedded in open-source project artefacts [33, 101]. In-IDE generation studies corroborate the same pattern at the individual level: developers remain responsible for inspecting and adapting generated code rather than accepting it outright [111], and increasing automation moves work towards specification, monitoring, verification, and repair rather than eliminating it [18].

3.3.3 Trust, Verification, and Reliance Behaviour

Effective human-AI interaction depends on calibrated reliance rather than generic trust. Acceptance of AI-generated suggestions is shaped by perceived quality,

contextual fit, controllability, effort awareness, and developers' capacity to verify outputs [14, 105, 23]. Trust-calibration mechanisms, including explicit indicators and controllability features, are something developers actively want rather than already have by default [105], and human judgements of code-generation value correlate with perceived accuracy and required effort in ways that offline metrics alone cannot explain [23]. Security-sensitive reliance introduces a sharper boundary condition for this calibration: developers using AI assistants can write more insecure code while simultaneously reporting higher confidence in that code [74], a pattern corroborated by qualitative evidence on security practices and concerns among AI-assisted developers [45] and by memorisation risks that complicate claims about LLM-based program repair [46]. Trust is therefore best understood as an interaction outcome, produced through inspectability, validation, domain-aware checks, and the explicit option to accept, reject, repair, or test, rather than as a static property of either the tool or the developer.

3.3.4 Developer Experience, Usability, and Perceived Value

Positive experience and continuance are important adoption signals, but the evidence is consistent that they do not establish engineering benefit on their own. Satisfaction, code volume, and perceived acceleration can coexist with inspection, debugging, confidentiality, accessibility, and integration costs [85, 72, 64]. Professional practitioners value LLM support while explicitly retaining responsibility for judgement and integration [85], and survey evidence from industry developers shows that continuance reflects perceived usefulness and fit rather than independently measured correctness or maintainability [72]. Usability concerns in this sub-theme are socio-technical rather than purely interface-level: accessibility-specific benefits and frictions have been documented for visually impaired developers using coding assistants [29], and AI Skill Threat was reported to affect 43–45% of surveyed developers, linking tool adoption directly to professional identity and thriving [35]. Taken together, this sub-theme indicates that perceived value is real but bounded: experience evidence improves understanding of adoption, but it cannot substitute for validation-based quality evidence.

3.3.5 Learning, Novices, and Educational Contexts

Educational evidence shows a persistent comprehension-performance gap: AI assistance can raise visible task progress for novices while leaving comprehension, testing ability, and durable transfer uncertain [76, 77, 88]. Novice programmers experience tools such as Copilot as usable and sometimes surprisingly anticipatory, but this fluency can obscure what learners actually understand [76], a divergence explicitly framed as a comprehension-performance gap in GenAI-assisted brownfield programming [77]. Working with unfamiliar, large code bases further increases the need for review, testing, and sense-making among student users [88]. Quantitative evidence on over-reliance is direct: 90% of students in one study expressed over-reliance concerns even while recognising the benefits of AI code completion [97]. This pattern is reinforced by evidence that prompt engineering itself must be learned as an interaction practice in introductory programming contexts [22], and that semester-long classroom collaboration with generative AI changes peer-like interaction dynamics in ways that require deliberate instructional scaffolding [60].

Educational use of AI-assisted programming should therefore actively force explanation, testing, reflection, and recovery strategies, so that visible task completion does not mask fragile understanding.

3.3.6 Robustness, Prompt Sensitivity, and Agentic Reliability

Developer-facing reliability is a system property: it depends on prompt quality, semantic stability, retrieval, localisation, patch generation, validation, runtime behaviour, and benchmark validity, rather than on single-run functional success alone [109,68,2,56]. Agentless decomposes LLM-based repair into localisation, patch generation, and validation stages, making each stage a potential reliability bottleneck in its own right [109], and self-repair is explicitly shown not to be a universal solution to code-generation failures [68]. Evaluation-harness evidence reinforces that LLM-guided programming requires dedicated infrastructure for assessing reliability rather than relying on benchmark pass rates alone [2], and end-to-end development benchmarking exposes multi-step coordination demands that isolated code-generation tasks do not capture [56]. A comprehensive comparison of seven general-purpose agent frameworks across software development, vulnerability detection, and program repair tasks shows that this coordination demand is not merely a cost to be managed but can actively undermine effectiveness: single-agent frameworks outperformed multi-agent frameworks on most tasks, with the multi-agent systems' longer reasoning trajectories and higher correction rates reflecting information loss and inter-agent error propagation rather than more thorough problem-solving [113]. Robustness claims become especially fragile under further scrutiny: self-consistency evaluation reveals that models can fail to reproduce their own correct outputs across semantically equivalent reformulations [63], and models can be shown to exploit test cases directly rather than solving the underlying task [117]. Robustness in human-AI interaction is therefore produced by the interaction between user specification, repository context, orchestration design, validation tooling, runtime constraints, and benchmark assumptions, not by any single one of these factors in isolation.

3.3.7 Identity, Accessibility, and Socio-Professional Continuity

Human-AI interaction also affects professional continuity by shaping identity, perceived skill threat, accessibility, inclusion, and cross-role communication. This sub-theme is less evenly evidenced than prompting, workflow, trust, or robustness, but the available evidence is informative. AI Skill Threat was found to affect 43–45% of surveyed developers, tying tool adoption directly to professional identity, thriving, and the meaning of developer expertise [35]. Generative AI coding assistants have also been shown to change the work of visually impaired developers by altering code navigation, explanation, and interaction, introducing frictions that studies of sighted users alone would not surface [29]. Organisational adoption research adds that uptake depends on task fit, confidentiality, and local context rather than on tool capability alone [21], and mixed-methods evidence shows that human-AI collaboration changes how developers coordinate assistance and oversight at the team level [93]. Cross-role evidence on reverse engineering and agile-epic evaluation extends this concern to artefact review and cross-role communication, though this evidence should be read as a cautious scope extension rather than as direct

corroboration of identity-level change [95]. Human–AI interaction is therefore not only a coding interface; it can reshape who participates in software development, how expertise is recognised, and how responsibility is negotiated.

3.3.8 *Synthesis*

Across all seven sub-themes, the HAI evidence base converges on a structural account in which interaction quality and trust calibration jointly determine whether generation efficiency is converted into dependable engineering value or displaced into hidden risk. Prompting, contextualisation, conversational repair, and representation choices shape the reliability and usefulness of AI-generated outputs across professional, educational, and agentic settings; trust calibration regulates whether developers accept, revise, test, or reject plausible but potentially incorrect or insecure suggestions. Cognitive redistribution connects these mechanisms by shifting developer effort away from direct code construction and towards supervision, orchestration, explanation, and decision-making. This pattern provides the empirical grounding in the corpus for treating interaction quality and trust calibration as the primary HAI constructs within the socio-technical equilibrium framework, and for understanding verification overhead, documented extensively in the CQTD dimension (Section 3.2), as something that is actively managed or amplified through the quality of human–AI interaction rather than fixed in advance. This connection is examined further in the Discussion.

3.4 Productivity

Productivity is the third-largest dimension in the corpus, accounting for 8.3% of studies ($n=26$; Section 3.1). AI assistance improves productivity most reliably when programming work is local, well specified, and readily verifiable, but the corpus does not support a universal acceleration claim. Controlled and benchmark-like studies show substantial generation efficiency: ChatGPT outperformed Stack Overflow on algorithmic and library-use tasks, including reported algorithmic task-score contrasts of 72.0 versus 0.0 and an average of 38.7 versus 5.0 in the relevant comparison [57]. Complexity then narrows the effect. ChatGPT-3.5 solved 92% of easy, 79% of medium, and 51% of hard coding problems in one complexity-stratified evaluation, with prompt engineering improving results but not removing the complexity gradient [54]. A separate difficulty-level study reported complex-problem performance at just 4.3%, indicating that decomposition and iteration are part of the work rather than optional refinements [112].

Brownfield and field evidence makes the productivity claim more conditional. In computing-student brownfield tasks, Copilot reduced completion time by 34.9%, increased solution progress by 50%, and reduced manual typing and web search by 10.6% and 11.6% respectively [89]. This supports a positive productivity mechanism: assistants can lower the cost of producing plausible edits, scaffolding searches and repetitive implementation steps. However, experienced open-source maintainers in a field experiment took 19% longer with AI assistance, despite predicting a 24% speed-up before the task and estimating afterwards that the tool had made them 20% faster [9]. This contradiction is not noise; it marks the difference between local

generation and situated software engineering, where repository context, testing, review, and integration can dominate raw output speed.

Enterprise evidence reinforces this interpretation. Developers using an enterprise AI code assistant modified outputs and treated checking as a normal part of use, so generated code functioned as a draft requiring human validation rather than as a finished artefact [108]. Across these studies, the relevant productivity unit is therefore not the suggestion, the line of code, or even the completed task in isolation: it is the net work required to move from generated candidate to accepted, maintainable project change. Productivity gains are strongest where the generated artefact is easy to inspect and weak where the artefact requires deep contextual reasoning, cross-file consistency, security judgement, or domain-specific integration.

A persistent limitation across this sub-literature is that productivity studies rarely measure speed, correctness, rework, and downstream integration within the same design. Some quantify completion time or task scores, while others observe acceptance, behaviour, or perceptions, making the evidence strong for conditional acceleration but weaker for total development productivity across real projects. The most defensible synthesis is that AI assistance increases generation efficiency, while its contribution to net productivity depends on whether verification overhead remains smaller than the construction effort saved. This positions productivity as the dimension in the corpus where the trade-off between generation efficiency and verification overhead, central to the socio-technical equilibrium framework, is most directly observable, a connection examined further in the Discussion.

3.5 Perception and Trust

Perception and Trust is the fourth-largest dimension in the corpus, accounting for 4.2% of studies ($n=13$; Section 3.1). The evidence shows that developers often regard AI coding assistants as useful, but trust is selective, contextual, and vulnerable to miscalibration. In a broad developer survey, 84% of 481 respondents reported using AI-based coding assistants, indicating that adoption is already normalised rather than experimental [87]. A separate survey of 207 developers found that 73% perceived productivity improvement from ChatGPT in software development [103]. These results establish perceived usefulness, but they do not by themselves establish dependable software outcomes; perception evidence becomes meaningful only when read against task difficulty, correctness evidence, and security risk.

Task difficulty is the clearest calibration boundary. Kuhail and colleagues reported 73% success on easy problems versus 10% on hard problems across 180 problems and 99 developers, which explains why favourable perceptions are more defensible for routine work than for complex development [49]. Acceptance conditions also matter: Suryavanshi and colleagues found that behavioural intention was associated with use behaviour, with a reported coefficient of $\beta = 0.419$, linking acceptance to facilitating conditions, perceived usefulness, perceived ease of use, and hedonic motivation [96]. Trust is therefore not a static attitude towards a tool; it is shaped by the local conditions that make use feasible, rewarding, and apparently safe.

The strongest caution comes from security and hallucination evidence. Developers report that generative AI can make their code less secure, shifting trust from the

generated artefact to inspection and security review [48]. Empirical secure-code assessment also shows uneven LLM performance across languages, so trust cannot be detached from the target language and vulnerability type [25]. Code hallucination evidence further shows that fluent code-like output can contain execution-relevant failures, making over-reliance dangerous when developers treat plausibility as correctness [99]. Trust calibration therefore requires repeated evidence about when the assistant is likely to be correct, not only a general belief that the assistant is helpful.

Interpretability-oriented evidence offers a constructive response to this miscalibration risk. A syntax-grounded explanation method that maps model confidence onto abstract-syntax-tree categories was found to support more accurate trust judgements than either a flat sequential display of token probabilities or no explanation at all: participants given the AST-based explanation were more likely to judge the underlying model as unreliable, and none judged it as highly reliable, compared with participants given only sequential probabilities, of whom 29% judged the model highly reliable despite the same underlying errors in every scenario [70]. This result reframes trust calibration as something that can be actively supported by interpretability tooling rather than left to a developer’s unaided judgement: explanations that expose where a model’s confidence is weak help pre-empt the kind of unwarranted confidence that the security and hallucination evidence above warns against.

Qualitative trust evidence makes this dynamic explicit. Developers define, grant, revise, and withdraw trust as they observe tool behaviour, task fit, and failure modes [82]. The key empirical pattern is thus a perception-reality gap: developers can experience acceleration and convenience while still needing to detect hallucinated dependencies, insecure code, weak reasoning, or contextually inappropriate suggestions. A persistent limitation is that many perception studies rely on self-report and are not consistently paired with objective code quality, security, or maintenance outcomes. Nevertheless, the combined evidence supports a strong synthesis: trustworthy AI-assisted programming is not produced by higher confidence or more frequent use, but by calibrated reliance under explicit evidence of correctness, task difficulty, and risk. This positions Perception and Trust as the dimension in the corpus most directly anchoring the trust-calibration construct of the socio-technical equilibrium framework, a connection examined further in the Discussion.

3.6 Cognitive Load and Human Factors

Cognitive Load and Human Factors is the fifth-largest dimension in the corpus, accounting for 2.9% of studies ($n=9$; Section 3.1). AI assistance changes the distribution of cognitive work rather than simply reducing it. The strongest behavioural evidence shows that developers spend substantial attention on interacting with and checking AI outputs: Mozannar and colleagues report that 51.5% of interaction time was spent in AI-related activities, with verification alone accounting for 22.4%, and suggestion acceptance around 34% [64]. This finding reframes the assistant as a generator of supervisory work: developers may write less code manually, but they must read, judge, compare, test, and decide whether the generated output is appropriate for the task.

A controlled experiment with 109 participants substantiates this redistribution directly by tracing how developers exercised, or failed to exercise, that supervisory work. Working with ChatGPT improved efficiency on simple, well-specified coding puzzles but not on a typical small-scale development task requiring developers to fix bugs in an existing codebase, and it did not improve solution quality in either setting [106]. Screen-history analysis of how participants interacted with the assistant explains this gap: roughly one third of participants on the development task tried to delegate the entire problem to ChatGPT without first attempting to understand the code themselves, and this group most often gave up or produced very low-quality solutions; by contrast, close to half of participants treated the assistant as a collaborator, alternating between asking it to explain unfamiliar code, proposing a fix, and pointing out concrete errors in its output before accepting a revision. This contrast indicates that cognitive redistribution is conditional on developers retaining and exercising comprehension, not an automatic consequence of using an AI assistant.

Educational and brownfield evidence supports the same redistribution pattern. In a self-paced CS1 setting, 33 novices produced 1,666 Codex interactions, showing extensive use of generation and explanation to progress through programming tasks [42]. This indicates construction support, but also a need for output interpretation and judgement. In brownfield programming, Qiao and colleagues report a comprehension-performance gap: GenAI assistance can support task completion more than program understanding when developers work with existing code [77]. The practical risk is that apparent completion may mask weak comprehension, reducing the developer’s ability to maintain, adapt, or validate the result later.

The student-performance evidence is mixed but converges on metacognitive risk. Rahe and Maalej report that programming students’ frequent code-generation use, direct copying, and low accuracy can weaken metacognitive control when use is not accompanied by review responsibility [79]. Jošt and colleagues similarly associate heavier reliance on generation and debugging with lower final performance, suggesting that short-term support may not translate into durable learning when comprehension is bypassed [40]. By contrast, Gardella and colleagues provide a positive counterpoint: novice programmers using AI code generation reported lower perceived workload and performance support [32]. The contrast is important because it shows that cognitive redistribution can be beneficial or harmful depending on whether supervision strengthens or replaces understanding.

Human-factors evidence also extends beyond performance to wellbeing. Practitioner accounts describe both relief from some work and increased mental demand, overload, fatigue, or blurred work-life boundaries when expectations and support are misaligned [62]. A persistent limitation is measurement heterogeneity: studies use time allocation, grades, task completion, comprehension, perceived workload, qualitative wellbeing, and related constructs, making pooled cognitive-load estimates inappropriate. The most defensible synthesis is therefore one of redistribution rather than reduction: AI assistance can lower local construction effort, but it increases supervisory, interpretive, and evaluative demands, and outcomes depend on whether developers retain enough comprehension to own the code they accept. This positions Cognitive Load and Human Factors as the dimension in the corpus that most directly evidences the cognitive-redistribution construct of the socio-technical equilibrium framework, a connection examined further in the Discussion.

3.7 Organisational Impact

Organisational Impact is the sixth-largest dimension in the corpus, accounting for 2.2% of studies ($n=7$; Section 3.1). At organisational scale, AI-assisted programming becomes a workflow, governance, and role-reconfiguration issue rather than merely an individual productivity tool. The strongest deployment evidence is CodeCompose at Meta: 16,000 developers used the system, 22% of suggestions were accepted, and approximately 8% of committed changed code was AI-authored [66]. These figures show that AI assistance can become part of production software engineering infrastructure, and they also show why organisational interpretation cannot stop at individual speed: once generated code enters shared repositories, teams must govern prompting practices, disclosure, review standards, monitoring, and responsibility for accepted output.

Enterprise case evidence indicates that successful adoption depends on contextual integration. The StackSpot AI case shows that a contextualised assistant required platform integration, repository context, and organisational learning loops, rather than a generic chatbot deployment [75]. Research software evidence points in the same direction: generative AI was used across development and maintenance in a research-intensive university, but adoption depended on local norms, reproducibility expectations, and institutional support [11]. These studies suggest that interaction quality at organisational level is produced by infrastructure and routines, not by tool availability alone.

Organisational impact also appears in professional roles and capability signals. Recruiter evidence indicates that AI-powered code-generation tools are changing hiring signals, with prompt skill, review judgement, and tool fluency becoming part of employability rather than peripheral tool knowledge [15]. Classroom collaboration evidence adds that generative AI can act as a collaborative partner in semester-long pair-programming settings, although outcomes remain contingent on task, partner configuration, and use practices [60]. Together, these findings show role reconfiguration: developers are evaluated not only by their ability to write code, but also by their ability to elicit, inspect, adapt, and govern AI-generated contributions.

A persistent limitation is that organisational evidence remains concentrated in case studies, deployments, surveys, and education-adjacent settings; longitudinal measurements of governance quality, review burden, accountability failures, maintenance outcomes, and organisational learning remain limited. Even so, the evidence is sufficient for a bounded claim: AI assistance scales when organisations embed it into repositories, review routines, competence models, and governance practices, and without those structures the same tools can fragment practice, obscure responsibility, and shift verification costs into informal work. This positions Organisational Impact as extending interaction quality and trust calibration from individuals to institutions, a connection examined further in the Discussion.

3.8 Economic Impact

Economic Impact is the smallest dimension in the corpus, accounting for 0.3% of studies ($n=1$; Section 3.1). Direct economic evidence on AI-assisted programming remains thin, and the corpus supports proxy-based economic claims more strongly than formal return-on-investment or total-cost claims. The best production proxy

is accepted AI-authored output at scale: CodeCompose at Meta reports 16,000 developers, 22% suggestion acceptance, and about 8% AI-authored changed code, indicating potential marginal savings in code authoring but not net economic value by itself [66]. Open-source activity evidence also suggests that generative AI exposure can alter development patterns, but activity is not the same as profitability, maintainability, or reduced total cost [55].

The hidden-cost evidence is more concrete. Smart-contract generation can produce compilable outputs while still requiring validity, correctness, security, and maintainability checks, converting apparent automation into validation labour [41]. In simple bug-fixing tasks, generated changes can increase repair burden, with reports of up to twice as many bugs as fixes unless prompts are constrained [38]. Software-development agents also leave substantial residual work: one evaluation reported 170 issues unresolved by all agents, alongside variation in patch patterns and quality [19]. These findings imply that economic value depends on the cost of checking and remediation, not only on the speed of generation.

Other proxies reinforce the same caution. AI coding proficiency varies sharply by technology stack, including reported differences of up to 84% between similar libraries and an 11.17% competitor-pair gap, so expected value depends on stack choice and project context [114]. Benchmark-based economic claims are also fragile when resolve rates are inflated by weak tests or leakage [3]. One FinTech total-cost-of-ownership comparison reports a lower unit cost with Copilot than junior-only staffing, roughly 61% return on investment, and a payback period of about 7.4 months, but it is a single peripheral, category-B study, so it should contextualise rather than anchor a general claim [81].

The defensible economic result is therefore cautious: AI assistance can reduce marginal effort in bounded settings, but net value depends on validation burden, unresolved work, compute and token costs, technology stack, developer expertise, governance, and adoption depth. This positions Economic Impact as the aggregate expression of the whole socio-technical equilibrium: savings become credible only when productivity proxies include validation, unresolved work, remediation, and governance costs, a connection examined further in the Discussion.

4 Discussion

4.1 The Productivity Paradox: Faster Development, Uncertain Outcomes

Productivity is the dimension in which the promise of AI-assisted programming is most directly tested, and the corpus evidence already presented in Section 3.4 shows that this promise is conditional rather than uniform. Gains concentrate in local, well-specified, easily inspected work, while experienced open-source maintainers in a controlled field experiment took 19% longer to complete real tasks with AI assistance, despite forecasting a 24% speed-up beforehand and believing, after the fact, that they had been 20% faster [9]. Read cross-dimensionally, this is not measurement noise but a timing mismatch: generation efficiency is realised the moment a candidate solution appears, whereas the verification overhead documented extensively in Section 3.2 (testing, review, security inspection, and contextual integration) is realised later and distributed across people and time. A productivity

paradox is the visible symptom of that mismatch: an immediate, locally observed benefit competing against a delayed, distributed cost.

This mismatch scales beyond the individual task. An econometric analysis of open-source repositories following the introduction of GitHub Copilot found that aggregate output rose, but the increase was driven mainly by peripheral, less-experienced contributors; the resulting quality risk fell on experienced core developers, whose own original-code productivity dropped by 19% as their review workload rose by 6.5% to keep newly generated code at repository standard [110]. Read alongside the workplace evidence above and the hidden-cost evidence already identified for Economic Impact (Section 3.8), this result reframes the productivity question: AI assistance does not so much create developer effort as relocate it, often from a broad, junior contributor base onto a narrower, more senior one whose time is the most expensive to spend on rework.

A closely related expression of the same paradox concerns whether benchmark performance predicts situated productivity. Section 3.2 already shows that stricter, hidden-test evaluation regimes can reduce apparent benchmark pass rates by as much as 28.9% [58]. A human-centred evaluation with 243 participants, testing seven underlying language models across both autocomplete and chat-based support, extends this finding from code correctness to programmer productivity directly: gains in benchmark performance are associated with real productivity gains, but disproportionately so, and programmers' stated preferences among the models tested do not track their measured performance, which removes self-report as a reliable substitute for outcome measurement [65]. The productivity promise of AI-assisted programming is therefore real, but systematically overstated by the two instruments, benchmarks and self-report, most often used to claim it.

The paradox is nonetheless a property of how validation is organised around generation, not a fixed ceiling on AI-assisted productivity. When interaction is restructured around test-driven clarification rather than one-shot generation, a controlled study allowing up to five rounds of user feedback reports an average absolute improvement of 38.43% in pass@1 accuracy across four language models and two datasets, together with a significant reduction in reported task-induced cognitive load [27]. This is the constructive mirror of the paradox documented above: verification overhead is not removed, but it is brought forward and made explicit during generation rather than discovered downstream after a defect has already shipped.

Taken together, these results support reading Productivity not as an independent outcome but as the most visible face of the trade-off between generation efficiency and verification overhead that anchors the socio-technical equilibrium framework proposed in this study. Whether a given deployment of AI-assisted programming registers as a productivity gain or a productivity paradox depends less on the raw capability of the underlying model than on whether verification is designed into the workflow, left implicit, or deferred onto a smaller group of developers further down the pipeline, a design choice that recurs, in different forms, throughout the remainder of this Discussion.

4.2 The Dominant Role of Human and Contextual Factors

The productivity paradox documented above is itself a symptom of a more general pattern that runs through every dimension examined in this study: the same underlying AI capability produces markedly different outcomes depending on who is using it and on what kind of task. Task difficulty is the most consistently documented moderator. Coding-problem benchmarks show success falling from 92% on easy problems to 51% on hard ones as complexity rises [54], complex-problem performance can drop to as little as 4.3% in a difficulty-stratified evaluation [112], and a developer survey already discussed in Section 3.5 reported 73% success on easy problems against 10% on hard ones [49]. Taken on its own, this body of evidence could suggest a single rule, that AI assistance helps on easy work and fails on hard work, but the corpus does not support a rule that simple once human factors and outcome type are taken into account.

Developer expertise interacts with task difficulty rather than substituting for it. In a controlled study of 109 participants, those who treated the assistant as a collaborator, alternating between requesting explanations, proposing fixes, and pointing out concrete errors before accepting a revision, produced markedly better outcomes than the roughly one third of participants who attempted to delegate the entire problem and most often gave up or produced very low-quality solutions [106]. Complementary evidence from computing-student brownfield tasks shows the same asymmetry from the opposite direction: stronger students used Copilot more selectively rather than more often, indicating that the benefit of assistance is conditional on already possessing enough judgement to know when to accept, adapt, or reject it [89]. AI assistance therefore amplifies existing capability more reliably than it substitutes for missing capability: it rewards developers who can convert a suggestion into a validated solution, and it can quietly increase risk for those who instead outsource the judgement that validation requires.

Context, however, is not a single dial that uniformly degrades outcomes as difficulty rises; its effect depends on which outcome is being measured. Security-focused evidence reverses the productivity-correctness gradient documented above: in a user study where participants solved programming problems with and without Copilot assistance, access to Copilot was associated with more secure solutions specifically on harder problems, with no measurable security effect on the easier problem and no disproportionate impact on particular vulnerability types [6]. A narrower domain produces a third pattern again: in a security-driven study of 58 student programmers restricted to low-level C pointer and array manipulation, AI-assisted participants produced critical security bugs at a rate no greater than 10% above the control group, suggesting that, within that constrained technical context, assistance neither degraded nor reliably improved security outcomes [84]. Read together with the technology-stack variation already noted for Economic Impact (Section 3.8), these results indicate that task difficulty, programming domain, and outcome type interact in ways that a single difficulty gradient cannot capture.

Human and contextual factors are therefore not nuisance variables to be averaged away when reporting the impact of AI-assisted programming; they are co-determinants of whether a given deployment amplifies engineering capability or amplifies risk. This reframes interaction quality, trust calibration, and cognitive redistribution, the constructs most directly implicated here, as conditional on who is interacting and under what task and domain conditions, rather than as

fixed properties of the underlying model. The next section examines the specific skills, prompting and validation practice, through which developers convert this conditional potential into a reliably positive outcome.

4.3 Prompting and Validation as Emerging Meta-Competencies

The human and contextual factors documented above are not abstract dispositions; in the corpus they cash out as two specific, observable practices: how developers prompt, and how they validate what comes back. Prompting functions as situated software work rather than a generic natural-language skill, requiring developers to assemble task goals, local context, and constraints into an interaction that progressively shapes model output (Section 3.3). An influential account of Copilot use frames this competency as bimodal: in acceleration mode, the developer already knows what to do and prompts to get there faster; in exploration mode, the developer is unsure how to proceed and prompts to discover options [7]. Knowing which mode applies, and prompting accordingly, is itself part of the skill, and the cost of getting it wrong is measurable: in one robustness study, only 27.43% of semantically equivalent paraphrases of the same Copilot prompt preserved the same correct output [61].

This sensitivity is not an unavoidable property of the technology; it is, to a meaningful degree, addressable through identifiable, transferable practice. A telemetry and qualitative analysis of an IDE-embedded editing feature found that frequent re-prompting signals a specific, recurring set of missing information, mostly about surrounding code, intent, and edit scope, and that automatically inferring this missing information from context raised edit correctness by 27% [67]. Technique-level evidence extends this to security outcomes specifically: a systematic evaluation of prompting strategies for secure code generation found that structured techniques, most notably Recursive Criticism and Improvement (RCI), reduced security weaknesses relative to unstructured prompting across the language models tested [100]. A detailed case study of building a 7,420-line framework through 107 prompts illustrates what this competency looks like at scale: most prompts were short, around 258 characters on average, and almost all were either bug-fix requests or feature requests rather than open-ended asks; the developer’s role consisted entirely of stating what was needed, checking whether the result behaved correctly, and sending short follow-up corrections, with no manual coding at all [28].

Validation is the second meta-competency, and it is no less specific than prompting. Eye-tracking and IDE-action evidence already discussed in Section 3.2 shows that effective validation means actively tracing logic and confirming correctness, not passively accepting output that merely looks complete [98]. Part of this skill is calibrating how much rework a given output actually needs: in one comparison, code that was incorrect but minimally modifiable occurred in 8.7% of outputs, and the time saved by correcting near-correct code relative to writing it from scratch ranged from 8.9% to 71.3% depending on how well that judgement was exercised [37]. The clearest demonstration that validation can be deliberately structured rather than left to instinct is the test-driven workflow discussed in Section 4.1: eliciting tests before accepting a solution, rather than after, raised pass@1 accuracy by an average of 38.43% [27].

Prompting and validation earn the label meta-competency because they operate above any specific language, framework, or domain: the same disciplined habits of specifying intent and checking output transfer across the tasks and contexts shown in Section 4.2 to interact so unevenly with raw AI capability. This is why the same underlying model can amplify one developer’s work and quietly damage another’s: the difference is not always domain expertise, but whether the developer has internalised a working practice for directing generation and confirming its results. Read this way, prompting and validation are the practical substrate through which interaction quality, trust calibration, and cognitive redistribution actually get exercised, day to day, rather than abstract properties of the socio-technical equilibrium framework. The next section considers what happens to the developer role once these competencies, rather than direct code authorship, become the primary measure of contribution.

5 Conclusion

References

1. Abbassi, A.A., Da Silva, L., Nikanjam, A., Khomh, F.: ReCatcher: Towards LLMs Regression Testing for Code Generation (2025). DOI 10.48550/ARXIV.2507.19390. URL <https://arxiv.org/abs/2507.19390>. Version Number: 1
2. Agarwal, A., Chan, A., Chandel, S., Jang, J., Miller, S., Moghaddam, R.Z., Mohylevskyy, Y., Sundaresan, N., Tufano, M.: Copilot Evaluation Harness: Evaluating LLM-Guided Software Programming (2024). DOI 10.48550/arXiv.2402.14261. URL <http://arxiv.org/abs/2402.14261>. ArXiv:2402.14261 [cs]
3. Aleithan, R., Xue, H., Mohajer, M.M., Nnorom, E., Uddin, G., Wang, S.: SWE-Bench+: Enhanced Coding Benchmark for LLMs (2024). DOI 10.48550/arXiv.2410.06992. URL <http://arxiv.org/abs/2410.06992>. ArXiv:2410.06992 [cs.SE]
4. Alrashedy, K., Aljasser, A., Tambwekar, P., Gombolay, M.: Can LLMs Patch Security Issues? (2024). DOI 10.48550/arXiv.2312.00024. URL <http://arxiv.org/abs/2312.00024>. ArXiv:2312.00024 [cs.CR]
5. Asare, O., Nagappan, M., Asokan, N.: Is GitHub’s Copilot as Bad as Humans at Introducing Vulnerabilities in Code? (2024). DOI 10.48550/arXiv.2204.04741. URL <http://arxiv.org/abs/2204.04741>. ArXiv:2204.04741 [cs.SE]
6. Asare, O., Nagappan, M., Asokan, N.: A User-centered Security Evaluation of Copilot. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, pp. 1–11. ACM, Lisbon Portugal (2024). DOI 10.1145/3597503.3639154. URL <https://dl.acm.org/doi/10.1145/3597503.3639154>
7. Barke, S., James, M.B., Polikarpova, N.: Grounded Copilot: How Programmers Interact with Code-Generating Models. Proceedings of the ACM on Programming Languages 7(OOPSLA1), 85–111 (2023). DOI 10.1145/3586030. URL <https://dl.acm.org/doi/10.1145/3586030>
8. Bayram, A., Menekse Dalveren Gonca Gokce, Derawi Mohammad: Comparative Analysis of AI Models for Python Code Generation: A HumanEval Benchmark Study. Applied Sciences 15(18), 9907 (2025). DOI 10.3390/app15189907. URL <https://www.proquest.com/scholarly-journals/comparative-analysis-ai-models-python-code/docview/3254469735/se-2?accountid=11643>. Place: Basel
9. Becker, J., Rush, N., Barnes, E., Rein, D.: Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity (2025). DOI 10.48550/arXiv.2507.09089. URL <http://arxiv.org/abs/2507.09089>. ArXiv:2507.09089 [cs]
10. Belozarov, V., Barclay, P.J., Sami, A.: Secure Coding with AI – From Detection to Repair (2026). DOI 10.48550/arXiv.2504.20814. URL <http://arxiv.org/abs/2504.20814>. ArXiv:2504.20814 [cs.SE]
11. Besser, S.A., Jensen, E.A., Katz, D.S.: How Generative AI is Shaping Research Software Development and Maintenance at a Research-Intensive University. Open Research Europe

- 6 (2026). DOI 10.12688/openreseurope.22009.1. URL <https://www.scopus.com/pages/publications/105034007932?origin=resultslist>
12. Borg, M., Hewett, D., Hagatulah, N., Couderc, N., Söderberg, E., Graham, D., Kini, U., Farley, D.: Echoes of AI: Investigating the Downstream Effects of AI Assistants on Software Maintainability (2026). DOI 10.48550/arXiv.2507.00788. URL <http://arxiv.org/abs/2507.00788>. ArXiv:2507.00788 [cs]
13. Bose, D.B.: From Prompts to Properties: Rethinking LLM Code Generation with Property-Based Testing. In: Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering, pp. 1660–1665. ACM, Clarion Hotel Trondheim Trondheim Norway (2025). DOI 10.1145/3696630.3728702. URL <https://dl.acm.org/doi/10.1145/3696630.3728702>
14. Brown, A., D’Angelo, S., Murillo, A., Jaspan, C., Green, C.: Identifying the Factors That Influence Trust in AI Code Completion. In: Proceedings of the 1st ACM International Conference on AI-Powered Software, pp. 1–9. ACM, Porto de Galinhas Brazil (2024). DOI 10.1145/3664646.3664757. URL <https://dl.acm.org/doi/10.1145/3664646.3664757>
15. Chen, A., Huo, T., Nam, Y., Port, D., Peruma, A.: The Impact of Generative AI-Powered Code Generation Tools on Software Engineer Hiring: Recruiters’ Experiences, Perceptions, and Strategies. In: Bui T.X. (ed.) Proc. Annu. Hawaii Int. Conf. Syst. Sci., pp. 7227–7236. IEEE Computer Society (2025). DOI 10.48550/arXiv.2409.00875. URL <https://www.scopus.com/pages/publications/105005154979?origin=resultslist>. Journal Abbreviation: Proc. Annu. Hawaii Int. Conf. Syst. Sci.
16. Chen, J., Huang, H., Lyu, Y., An, J., Shi, J., Yang, C., Zhang, T., Tian, H., Li, Y., Li, Z., Zhou, X., Hu, X., Lo, D.: SecureVibeBench: Evaluating Secure Coding Capabilities of Code Agents with Realistic Vulnerability Scenarios (2026). DOI 10.48550/arXiv.2509.22097. URL <http://arxiv.org/abs/2509.22097>. ArXiv:2509.22097 [cs] v3, accessed 2026-04-15
17. Chen, J., Li, Z., Hu, X., Xia, X.: NLPerturbator: Studying the Robustness of Code LLMs to Natural Language Variations. ACM Transactions on Software Engineering and Methodology **35**(4), 1–20 (2026). DOI 10.1145/3745764. URL <https://dl.acm.org/doi/10.1145/3745764>
18. Chen, V., Talwalkar, A., Brennan, R., Neubig, G.: Code with Me or for Me? How Increasing AI Automation Transforms Developer Workflows (2025). DOI 10.48550/arXiv.2507.08149. URL <http://arxiv.org/abs/2507.08149>. ArXiv:2507.08149 [cs]
19. Chen, Z., Jiang, L.: Evaluating Software Development Agents: Patch Patterns, Code Quality, and Issue Complexity in Real-World GitHub Scenarios (2024). DOI 10.48550/arXiv.2410.12468. URL <http://arxiv.org/abs/2410.12468>. ArXiv:2410.12468 [cs]
20. Cotroneo, D., De Luca, R., Liguori, P.: DeVAIC: A Tool for Security Assessment of AI-Generated Code. Information and Software Technology **177**, 107572 (2025). DOI 10.1016/j.infsof.2024.107572. URL <https://linkinghub.elsevier.com/retrieve/pii/S0950584924001770>
21. Davila, N., Wiese, I., Steinmacher, I., Lucio Da Silva, L., Kawamoto, A., Favaro, G.J.P., Nunes, I.: An Industry Case Study on Adoption of AI-based Programming Assistants. In: Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Practice, pp. 92–102. ACM, Lisbon Portugal (2024). DOI 10.1145/3639477.3643648. URL <https://dl.acm.org/doi/10.1145/3639477.3643648>
22. Denny, P., Kumar, V., Giacaman, N.: Conversing with Copilot: Exploring Prompt Engineering for Solving CS1 Problems Using Natural Language. In: Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1, pp. 1136–1142. ACM, Toronto ON Canada (2023). DOI 10.1145/3545945.3569823. URL <https://dl.acm.org/doi/10.1145/3545945.3569823>
23. Dibia, V., Fournay, A., Bansal, G., Poursabzi-Sangdeh, F., Liu, H., Amershi, S.: Aligning Offline Metrics and Human Judgments of Value for Code Generation Models. In: Findings of the Association for Computational Linguistics: ACL 2023, pp. 8516–8528. Association for Computational Linguistics, Toronto, Canada (2023). DOI 10.18653/v1/2023.findings-acl.540. URL <https://aclanthology.org/2023.findings-acl.540>
24. Dou, S., Jia, H., Wu, S., Zheng, H., Wu, M., Tao, Y., Zhang, M., Chai, M., Fan, J., Xi, Z., Zheng, R., Wu, Y., Wen, M., Gui, T., Zhang, Q., Qiu, X., Huang, X.: What’s Wrong with Your Code Generated by Large Language Models? An Extensive Study. Science China Information Sciences **69**(1), 112107 (2026). DOI 10.1007/s11432-025-4632-8. URL <https://doi.org/10.1007/s11432-025-4632-8>
25. Dozono, K., Gasiba, T.E., Stocco, A.: Large Language Models for Secure Code Assessment: A Multi-Language Empirical Study (2026). DOI 10.48550/arXiv.2408.06428. URL <http://arxiv.org/abs/2408.06428>. ArXiv:2408.06428 [cs]

26. Esfahani, A.M., Kahani, N., Ajila, S.A.: Understanding Defects in Generated Codes by Language Models. In: 2024 34th International Conference on Collaborative Advances in Software and COmputiNg (CASCON), pp. 1–10 (2024). DOI 10.1109/CASCON62161.2024.10837857. Journal Abbreviation: 2024 34th International Conference on Collaborative Advances in Software and COmputiNg (CASCON)
27. Fakhoury, S., Naik, A., Sakkas, G., Chakraborty, S., Lahiri, S.K.: LLM-based test-driven interactive code generation: User study and empirical evaluation. arXiv.org (2024). DOI 10.48550/arxiv.2404.10100. URL <https://ieeexplore.ieee.org/abstract/document/10606356/>
28. Fayed, M.S., Fayed, A.S.: Prompt-Driven Development with Claude Code: Developing a TUI Framework for the Ring Programming Language. *Electronics* **15**(4), 903 (2026). DOI 10.3390/electronics15040903. URL <https://www.proquest.com/scholarly-journals/prompt-driven-development-with-claude-code/docview/3307482518/se-2?accountid=11643>. Place: Basel
29. Flores-Saviaga, C., Hanrahan, B.V., Imteyaz, K., Clarke, S., Savage, S.: The Impact of Generative AI Coding Assistants on Developers Who Are Visually Impaired. In: Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems, pp. 1–17. ACM, Yokohama Japan (2025). DOI 10.1145/3706598.3714008. URL <https://dl.acm.org/doi/10.1145/3706598.3714008>
30. Fu, Y., Liang, P., Tahir, A., Li, Z., Shahin, M., Yu, J., Chen, J.: Security Weaknesses of Copilot-Generated Code in GitHub Projects: An Empirical Study. *ACM Transactions on Software Engineering and Methodology* **34**(8), 1–34 (2025). DOI 10.1145/3716848. URL <https://dl.acm.org/doi/10.1145/3716848>
31. Gao, S., Li, E.J., Lam, M.H., Xiao, J., Wan, Y., Wang, C., Tik, N.M., Lyu, M.R.: TREAT: A Code LLMs Trustworthiness / Reliability Evaluation and Testing Framework (2025). DOI 10.48550/arXiv.2510.17163. URL <http://arxiv.org/abs/2510.17163>. ArXiv:2510.17163 [cs]
32. Gardella, N., Pettit, R., Riggs, S.L.: Performance, Workload, Emotion, and Self-Efficacy of Novice Programmers Using AI Code Generation. In: Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1, pp. 290–296. ACM, Milan Italy (2024). DOI 10.1145/3649217.3653615. URL <https://dl.acm.org/doi/10.1145/3649217.3653615>
33. Grewal, B., Lu, W., Nadi, S., Bezemer, C.P.: Analyzing Developer Use of ChatGPT Generated Code in Open Source GitHub Projects. In: Proceedings of the 21st International Conference on Mining Software Repositories, pp. 157–161. ACM, Lisbon Portugal (2024). DOI 10.1145/3643991.3645072. URL <https://dl.acm.org/doi/10.1145/3643991.3645072>
34. Hao, H., Hasan, K.A., Qin, H., Macedo, M., Tian, Y., Ding, S.H.H., Hassan, A.E.: An Empirical Study on Developers’ Shared Conversations with ChatGPT in GitHub Pull Requests and Issues. *Empirical Software Engineering* **29**(6), 150 (2024). DOI 10.1007/s10664-024-10540-x. URL <https://www.proquest.com/scholarly-journals/empirical-study-on-developers-shared/docview/3105555573/se-2?accountid=11643>. Place: Dordrecht
35. Hicks, C.M., Lee, C.S., Foster-Marks, K.: The New Developer: AI Skill Threat, Identity Change & Developer Thriving in the Transition to AI-Assisted Software Development (2024). DOI 10.31234/osf.io/2gej5. URL https://osf.io/2gej5_v1
36. Honarvar, S., Wilk, M.v.d., Donaldson, A.: Turbulence: Systematically and Automatically Testing Instruction-Tuned Large Language Models for Code (2025). DOI 10.48550/arXiv.2312.14856. URL <http://arxiv.org/abs/2312.14856>. ArXiv:2312.14856 [cs]
37. Idrisov, B., Schlippe, T.: Program Code Generation with Generative AIs. *Algorithms* **17**(2), 62 (2024). DOI 10.3390/a17020062. URL <https://www.proquest.com/scholarly-journals/program-code-generation-with-generative-ais/docview/2930477147/se-2?accountid=11643>
38. Jesse, K., Ahmed, T., Devanbu, P.T., Morgan, E.: Large Language Models and Simple, Stupid Bugs (2023). DOI 10.48550/arXiv.2303.11455. URL <http://arxiv.org/abs/2303.11455>. ArXiv:2303.11455 [cs]
39. Jiang, M., Jain, A., Zorek, S., Jermaine, C.: SIMCOPILOT: Evaluating Large Language Models for Copilot-Style Code Generation (2025). DOI 10.48550/arXiv.2505.21514. URL <http://arxiv.org/abs/2505.21514>. ArXiv:2505.21514 [cs]
40. Jošt, G., Taneski, V., Karakatič, S.: The Impact of Large Language Models on Programming Education and Student Learning Outcomes. *Applied Sciences* (2024). DOI 10.3390/app14104115. URL <https://www.mdpi.com/2076-3417/14/10/4115>

41. Karanjai, R., Li, E., Xu, L., Shi, W.: Who is Smarter? An Empirical Study of AI-Based Smart Contract Creation. In: 2023 5th Conference on Blockchain Research & Applications for Innovative Networks and Services (BRAINS), pp. 1–8 (2023). DOI 10.1109/BRAINS59668.2023.10316829. Journal Abbreviation: 2023 5th Conference on Blockchain Research & Applications for Innovative Networks and Services (BRAINS)
42. Kazemitabaar, M., Hou, X., Henley, A., Ericson, B., Weintrop, D., Grossman, T.: How Novices Use LLM-based Code Generators to Solve CS1 Coding Tasks in a Self-Paced Learning Environment. arXiv.org (2023). DOI 10.48550/arxiv.2309.14049. URL <https://dl.acm.org/doi/abs/10.1145/3631802.3631806>
43. Khojah, R., Mohamad, M., Leitner, P., Neto, F.G.d.O.: Beyond Code Generation: An Observational Study of ChatGPT Usage in Software Engineering Practice. arXiv.org (2024). DOI 10.48550/arxiv.2404.14901. URL <https://dl.acm.org/doi/abs/10.1145/3660788>
44. Kitchenham, B., Charters, S.: Guidelines for performing Systematic Literature Reviews in Software Engineering. Tech. rep., Technical report, EBSE Technical Report EBSE-2007-01 (2007). URL https://www.researchgate.net/publication/302924724_Guidelines_for_performing_Systematic_Literature_Reviews_in_Software_Engineering
45. Klemmer, J.H., Horstmann, S.A., Patnaik, N., Ludden, C., Burton, C., Powers, C., Massacci, F., Rahman, A., Votipka, D., Lipford, H.R., Rashid, A., Naiakshina, A., Fahl, S.: Using AI Assistants in Software Development: A Qualitative Study on Security Practices and Concerns. In: Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, pp. 2726–2740. ACM, Salt Lake City UT USA (2024). DOI 10.1145/3658644.3690283. URL <https://dl.acm.org/doi/10.1145/3658644.3690283>
46. Kong, J., Xie, X., Liu, S.: Demystifying Memorization in LLM-Based Program Repair via a General Hypothesis Testing Framework. Proceedings of the ACM on Software Engineering **2**(FSE), 2712–2734 (2025). DOI 10.1145/3729390. URL <https://dl.acm.org/doi/10.1145/3729390>
47. Kozak, M., Moghaddam, R.Z., Sivaraman, S.: When Developer Aid Becomes Security Debt: A Systematic Analysis of Insecure Behaviors in LLM Coding Agents. arXiv.org (2025). DOI 10.48550/arxiv.2507.09329. URL <https://arxiv.org/abs/2507.09329>
48. Kudriavtseva, A., Hotak, N.A., Gadyatskaya, O.: My Code Is Less Secure with Gen AI: Surveying Developers’ Perceptions of the Impact of Code Generation Tools on Security. In: Proc ACM Symp Appl Computing, pp. 1637–1646. Association for Computing Machinery (2025). DOI 10.1145/3672608.3707778. URL <https://www.scopus.com/pages/publications/105006445991?origin=resultslist>. Journal Abbreviation: Proc ACM Symp Appl Computing
49. Kuhail, M.A., Mathew, S.S., Khalil, A., Berengueres, J., Shah, S.J.H.: “Will I be replaced?” Assessing ChatGPT’s effect on software development and programmer perceptions of AI tools. Science of Computer Programming **235**, 103111 (2024). DOI 10.1016/j.scico.2024.103111. URL <https://linkinghub.elsevier.com/retrieve/pii/S0167642324000340>
50. Larbi, M., Akli, A., Papadakis, M., Bouyousfi, R., Cordy, M., Sarro, F., Traon, Y.L.: When Prompts Go Wrong: Evaluating Code Model Robustness to Ambiguous, Contradictory, and Incomplete Task Descriptions (2025). DOI 10.48550/arXiv.2507.20439. URL <http://arxiv.org/abs/2507.20439> [cs]
51. Latendresse, J., Khatoonabadi, S., Shihab, E.: How Robust are LLM-Generated Library Imports? An Empirical Study using Stack Overflow (2025). DOI 10.48550/ARXIV.2507.10818. URL <https://arxiv.org/abs/2507.10818>. Version Number: 1
52. Li, B., Wu, W., Tang, Z., Shi, L., Yang, J., Li, J., Yao, S., Qian, C., Hui, B., Zhang, Q., Yu, Z., Du, H., Yang, P., Lin, D., Peng, C., Chen, K.: Prompting Large Language Models to Tackle the Full Software Development Lifecycle: A Case Study (2024). DOI 10.48550/arXiv.2403.08604. URL <http://arxiv.org/abs/2403.08604>. ArXiv:2403.08604 [cs.CL]
53. Li, J., Li, J., Li, G., Chang, Y., Wu, Y.: Refining Critical Thinking in LLM Code Generation: A Faulty Premise-based Evaluation Framework (2025). DOI 10.48550/arXiv.2508.03622. URL <http://arxiv.org/abs/2508.03622>. ArXiv:2508.03622 [cs.AI]
54. Li, M., Krishnamachari, B.: Evaluating ChatGPT-3.5 Efficiency in Solving Coding Problems of Different Complexity Levels: An Empirical Analysis (2024). DOI 10.48550/arXiv.2411.07529. URL <http://arxiv.org/abs/2411.07529>. ArXiv:2411.07529 [cs]
55. Li, X., Costello, F.J., Kim, K.: Exploring Altered Open Source Software Development Patterns in a Time of Generative AI (2025). DOI 10.2139/ssrn.5100609. URL https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5100609. Empirical study; OSS development; generative AI; GitHub-based behavioral analysis

56. Liu, J., Huang, C., Guan, Z., Lei, W., Deng, Y.: E2Edev: Benchmarking Large Language Models in End-to-End Software Development Task (2026). DOI 10.48550/arXiv.2510.14509. URL <http://arxiv.org/abs/2510.14509>. ArXiv:2510.14509 [cs]
57. Liu, J., Tang, X., Li, L., Chen, P., Liu, Y.: ChatGPT vs. Stack Overflow: An Exploratory Comparison of Programming Assistance Tools. In: 2023 IEEE 23rd International Conference on Software Quality, Reliability, and Security Companion (QRS-C), pp. 364–373. IEEE, Chiang Mai, Thailand (2023). DOI 10.1109/QRS-C60940.2023.00105. URL <https://ieeexplore.ieee.org/document/10430054/>
58. Liu, J., Xia, C.S., Wang, Y., Zhang, L.: Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. In: A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, S. Levine (eds.) *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, pp. 21558–21572. Curran Associates, Inc., New Orleans, Louisiana, USA (2023). DOI 10.48550/arxiv.2305.01210. URL https://proceedings.neurips.cc/paper_files/paper/2023. NeurIPS Proceedings
59. Liu, Y., Widyasari, R., Zhao, Y., Irsan, I.C., Lo, D.: Debt Behind the AI Boom: A Large-Scale Empirical Study of AI-Generated Code in the Wild (2026). DOI 10.48550/arXiv.2603.28592. URL <http://arxiv.org/abs/2603.28592>. ArXiv:2603.28592 [cs]
60. Lyu, W., Wang, Y., Sun, Y., Zhang, Y.: Will Your Next Pair Programming Partner Be Human? An Empirical Evaluation of Generative AI as a Collaborative Teammate in a Semester-Long Classroom Setting. In: *Proceedings of the Twelfth ACM Conference on Learning @ Scale*, pp. 83–94. ACM, Palermo Italy (2025). DOI 10.1145/3698205.3729544. URL <https://dl.acm.org/doi/10.1145/3698205.3729544>
61. Mastropaolo, A., Pascarella, L., Guglielmi, E., Ciniselli, M., Scalabrino, S., Oliveto, R., Bavota, G.: On the Robustness of Code Generation Techniques: An Empirical Study on GitHub Copilot (2023). DOI 10.48550/arXiv.2302.00438. URL <http://arxiv.org/abs/2302.00438>. ArXiv:2302.00438 [cs]
62. Meem, F.N., Johnson, B.: Investigating the Impact of AI-Assisted Tools on Software Practitioner Well-Being. In: *Proc. of IEEE Symp. Vis. Lang. Hum.-Cent. Comput., VL/HCC*, pp. 447–449. IEEE Computer Society (2025). DOI 10.1109/VL-HCC65237.2025.00068. URL <https://www.scopus.com/pages/publications/105031773617?origin=resultslist>. Journal Abbreviation: *Proc. of IEEE Symp. Vis. Lang. Hum.-Cent. Comput., VL/HCC*
63. Min, M.J., Ding, Y., Buratti, L., Pujar, S., Kaiser, G., Jana, S., Ray, B.: Beyond Accuracy: Evaluating Self-Consistency of Code Large Language Models with IdentityChain. In: B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, Y. Sun (eds.) *International Conference on Learning Representations*, vol. 2024, pp. 5454–5469 (2024). URL https://proceedings.iclr.cc/paper_files/paper/2024/file/16371a9d5fed65d6d78ca3a7fa6e598c-Paper-Conference.pdf
64. Mozannar, H., Bansal, G., Fourney, A., Horvitz, E.: Reading Between the Lines: Modeling User Behavior and Costs in AI-Assisted Programming. In: *Proceedings of the CHI Conference on Human Factors in Computing Systems*, pp. 1–16. ACM, Honolulu HI USA (2024). DOI 10.1145/3613904.3641936. URL <https://dl.acm.org/doi/10.1145/3613904.3641936>
65. Mozannar, H., Chen, V., Alsobay, M., Das, S., Zhao, S., Wei, D., Nagireddy, M., Sattigeri, P., Talwalkar, A., Sontag, D.: The RealHumanEval: Evaluating Large Language Models’ Abilities to Support Programmers (2024). DOI 10.48550/arXiv.2404.02806. URL <http://arxiv.org/abs/2404.02806>. ArXiv:2404.02806 [cs]
66. Murali, V., Maddila, C., Ahmad, I., Bolin, M., Cheng, D., Ghorbani, N., Fernandez, R., Nagappan, N., Rigby, P.C.: AI-Assisted Code Authoring at Scale: Fine-Tuning, Deploying, and Mixed Methods Evaluation. *Proceedings of the ACM on Software Engineering 1(FSE)*, 1066–1085 (2024). DOI 10.1145/3643774. URL <https://dl.acm.org/doi/10.1145/3643774>
67. Nam, D., Omran, A., Murillo, A., Thakur, S., Araujo, A., Blistein, M., Frömmgen, A., Hellendoorn, V., Chandra, S.: Understanding and Supporting How Developers Prompt for LLM-powered Code Editing in Practice (2025). DOI 10.48550/arXiv.2504.20196. URL <http://arxiv.org/abs/2504.20196>. ArXiv:2504.20196 [cs]
68. Olausson, T.X., Inala, J.P., Wang, C., Gao, J., Solar-Lezama, A.: Is Self-Repair A Silver Bullet for Code Generation? In: *The Twelfth International Conference on Learning Representations (ICLR 2024)*, ICLR 2024. International Conference on Learning Representations (2024). DOI 10.48550/arxiv.2306.09896. URL <https://arxiv.org/abs/2306.09896>

69. Page, M.J., McKenzie, J.E., Bossuyt, P.M., Boutron, I., Hoffmann, T.C., Mulrow, C.D., Shamseer, L., Tetzlaff, J.M., Akl, E.A., Brennan, S.E., Chou, R., Glanville, J., Grimshaw, J.M., Hróbjartsson, A., Lalu, M.M., Li, T., Loder, E.W., Mayo-Wilson, E., McDonald, S., McGuinness, L.A., Stewart, L.A., Thomas, J., Tricco, A.C., Welch, V.A., Whiting, P., Moher, D.: The PRISMA 2020 Statement: An Updated Guideline for Reporting Systematic Reviews. *BMJ* **372**, n71 (2021). DOI 10.1136/bmj.n71. URL <https://www.bmj.com/content/372/bmj.n71.abstract>
70. Palacio, D.N., Rodriguez-Cardenas, D., Velasco, A., Khatri, D., Moran, K., Poshyanyk, D.: Towards More Trustworthy and Interpretable LLMs for Code through Syntax-Grounded Explanations. *ACM Transactions on Software Engineering and Methodology* p. 3771282 (2025). DOI 10.1145/3771282. URL <http://arxiv.org/abs/2407.08983>. ArXiv:2407.08983 [cs]
71. Pan, R., Ibrahimzada, A.R., Krishna, R., Sankar, D., Wassi, L.P., Merler, M., Sobolev, B., Pavuluri, R., Sinha, S., Jabbarvand, R.: Lost in Translation: A Study of Bugs Introduced by Large Language Models while Translating Code. In: *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, pp. 1–13 (2024). DOI 10.1145/3597503.3639226. URL <http://arxiv.org/abs/2308.03109>. ArXiv:2308.03109 [cs.SE]
72. Park, E.: Continuation Use of AI Coding Assistants Among South Korean Industry Developers: A Survey Case Study with Large Language Models. *Empirical Software Engineering* (2025). DOI 10.1007/s10664-025-10730-1. URL <https://link.springer.com/article/10.1007/s10664-025-10730-1>
73. Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., Karri, R.: Asleep at the Keyboard? Assessing the Security of GitHub Copilot’s Code Contributions. *Communications of the ACM* **68**(2), 96–105 (2025). DOI 10.1145/3610721. URL <https://dl.acm.org/doi/10.1145/3610721>. Originally published in: *Proceedings of the 2022 IEEE Symposium on Security and Privacy (SP)*
74. Perry, N., Srivastava, M., Kumar, D., Boneh, D.: Do Users Write More Insecure Code with AI Assistants? In: *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, pp. 2785–2799. ACM, Copenhagen Denmark (2023). DOI 10.1145/3576915.3623157. URL <https://dl.acm.org/doi/10.1145/3576915.3623157>
75. Pinto, G., De Souza, C., Neto, J., Souza, A., Gotto, T., Monteiro, E.: Lessons from Building StackSpot AI: A Contextualized AI Coding Assistant. In: *ACM Int. Conf. Proc. Ser.*, pp. 408–417. Association for Computing Machinery (2024). DOI 10.1145/3639477.3639751. URL <https://www.scopus.com/pages/publications/85195305220?origin=resultslist>. Journal Abbreviation: *ACM Int. Conf. Proc. Ser.*
76. Prather, J., Reeves, B.N., Denny, P., Becker, B.A., Leinonen, J., Luxton-Reilly, A., Powell, G., Finnie-Ansley, J., Santos, E.A.: “It’s Weird That it Knows What I Want”: Usability and Interactions with Copilot for Novice Programmers. *ACM Transactions on Computer-Human Interaction* **31**(1), 1–31 (2024). DOI 10.1145/3617367. URL <https://dl.acm.org/doi/10.1145/3617367>
77. Qiao, Y., Hundhausen, C., Haque, S., Shihab, M.I.H.: Comprehension-Performance Gap in GenAI-Assisted Brownfield Programming: A Replication and Extension (2025). DOI 10.48550/arXiv.2511.02922. URL <http://arxiv.org/abs/2511.02922>. ArXiv:2511.02922 [cs]
78. Rabbi, M.F., Champa, A.I., Zibran, M.F., Islam, M.R.: AI Writes, We Analyze: The ChatGPT Python Code Saga. In: *Proceedings of the 21st International Conference on Mining Software Repositories*, pp. 177–181. ACM, Lisbon Portugal (2024). DOI 10.1145/3643991.3645076. URL <https://dl.acm.org/doi/10.1145/3643991.3645076>
79. Rahe, C., Maalej, W.: How Do Programming Students Use Generative AI? *Proceedings of the ACM on Software Engineering* **2**(FSE), 978–1000 (2025). DOI 10.1145/3715762. URL <https://dl.acm.org/doi/10.1145/3715762>
80. Res, J., Homoliak, I., Perešini, M., Smrčka, A., Malinka, K., Hanacek, P.: Enhancing Security of AI-Based Code Synthesis with GitHub Copilot via Cheap and Efficient Prompt-Engineering (2024). DOI 10.48550/arXiv.2403.12671. URL <http://arxiv.org/abs/2403.12671>. ArXiv:2403.12671 [cs]
81. Rodríguez, J.A.: Comparative Evaluation of Total Cost of Ownership (TCO) in Software Development for a FinTech Company with Operations in Costa Rica: Junior Human Capital vs. GitHub Copilot (2025). URL https://papers.ssrn.com/sol3/papers.cfm?abstract_id=6266838
82. Sabouri, S., Eibl, P., Zhou, X., Ziyadi, M., Medvidovic, N., Lindemann, L., Chattopadhyay, S.: Trust Dynamics in AI-Assisted Development: Definitions, Factors, and Implications.

- In: Proc Int Conf Software Eng, pp. 1678–1690. IEEE Computer Society (2025). DOI 10.1109/ICSE55347.2025.00199. URL <https://www.scopus.com/pages/publications/105010311765?origin=resultslist>. Journal Abbreviation: Proc Int Conf Software Eng
83. Sajadi, A., Damevski, K., Chatterjee, P.: How Safe Are AI-Generated Patches? A Large-scale Study on Security Risks in LLM and Agentic Automated Program Repair on SWE-bench (2025). DOI 10.48550/arXiv.2507.02976. URL <http://arxiv.org/abs/2507.02976>. ArXiv:2507.02976 [cs.CR]
 84. Sandoval, G., Pearce, H., Nys, T., Karri, R., Garg, S., Dolan-Gavitt, B.: Lost at C: A User Study on the Security Implications of Large Language Model Code Assistants (2022). URL <https://www.usenix.org/conference/usenixsecurity23/presentation/sandoval>
 85. Santos, I., Magalhaes, C., Santos, R.d.S.: Model-Assisted and Human-Guided: Perceptions and Practices of Software Professionals Using LLMs for Coding (2025). DOI 10.48550/ARXIV.2510.09058. URL <https://arxiv.org/abs/2510.09058>. Version Number: 1
 86. Schreiber, M., Tippe, P.: Security Vulnerabilities in AI-Generated Code: A Large-Scale Analysis of Public GitHub Repositories. pp. 153–172 (2026). DOI 10.1007/978-981-95-3537-8_9. URL <http://arxiv.org/abs/2510.26103>. ArXiv:2510.26103 [cs]
 87. Sergeyuk, A., Golubev, Y., Bryksin, T., Ahmed, I.: Using AI-Based Coding Assistants in Practice: State of Affairs, Perceptions, and Ways Forward. Information and Software Technology **178**, 107610 (2025). DOI 10.1016/j.infsof.2024.107610. URL <http://arxiv.org/abs/2406.07765>. ArXiv:2406.07765 [cs]
 88. Shah, A., Chernova, A., Tomson, E., Porter, L., Griswold, W.G., Soosai Raj, A.G.: Students’ Use of GitHub Copilot for Working with Large Code Bases. In: Proceedings of the 56th ACM Technical Symposium on Computer Science Education V. 1, pp. 1050–1056. ACM, Pittsburgh PA USA (2025). DOI 10.1145/3641554.3701800. URL <https://dl.acm.org/doi/10.1145/3641554.3701800>
 89. Shihab, M.I.H., Hundhausen, C., Tariq, A., Haque, S., Qiao, Y., Mulanda, B.W.: The Effects of GitHub Copilot on Computing Students’ Programming Effectiveness, Efficiency, and Processes in Brownfield Coding Tasks. In: Proceedings of the 2025 ACM Conference on International Computing Education Research V.1, pp. 407–420. ACM, Charlottesville USA (2025). DOI 10.1145/3702652.3744219. URL <https://dl.acm.org/doi/10.1145/3702652.3744219>
 90. Shirafuji, A., Watanobe, Y., Ito, T., Morishita, M., Nakamura, Y., Oda, Y., Suzuki, J.: Exploring the Robustness of Large Language Models for Solving Programming Problems (2023). DOI 10.48550/arXiv.2306.14583. URL <http://arxiv.org/abs/2306.14583>. ArXiv:2306.14583 [cs]
 91. Siddiq, M.L., Roney, L., Zhang, J., Santos, J.C.D.S.: Quality Assessment of ChatGPT Generated Code and their Use by Developers. In: Proceedings of the 21st International Conference on Mining Software Repositories, pp. 152–156. ACM, Lisbon Portugal (2024). DOI 10.1145/3643991.3645071. URL <https://dl.acm.org/doi/10.1145/3643991.3645071>
 92. Siddiq, M.L., Santos, J.C.S.: Generate and Pray: Using SALLMS to Evaluate the Security of LLM Generated Code. arXiv.org (2023). DOI 10.48550/arxiv.2311.00889. URL <https://lsiddiqsunny.github.io/public/2311.00889.pdf>
 93. Stray, V., Barbala, A., Wivestad, V.T.: Human-AI Collaboration in Software Development: A Mixed-Methods Study of Developers’ Use of GitHub Copilot and ChatGPT. In: Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering, pp. 1325–1332. ACM, Clarion Hotel Trondheim Trondheim Norway (2025). DOI 10.1145/3696630.3730566. URL <https://dl.acm.org/doi/10.1145/3696630.3730566>
 94. Stray, V., Moe, N.B., Ganeshan, N., Kobbenes, S.: Generative AI and Developer Workflows: How GitHub Copilot and ChatGPT Influence Solo and Pair Programming (2025). DOI 10.24251/HICSS.2025.883. URL <https://hdl.handle.net/10125/109735>
 95. Surk, E., Menekşe Dalveren, G.G., Mohammad, D.: Reducing Information Asymmetry in Software Product Management: An LLM-Based Reverse Engineering Framework. Applied Sciences **16**(6), 2801 (2026). DOI 10.3390/app16062801. URL <https://www.proquest.com/scholarly-journals/reducing-information-asymmetry-software-product/docview/3322201304/se-2?accountid=11643>. Place: Basel
 96. Suryavanshi, P., Kapse, M., Sharma, V.: Integrating ChatGPT into Software Development: Valuating Acceptance and Utilisation Among Developers. Australasian Accounting Business & Finance Journal **19**(1), 96–117 (2025). URL <https://www.proquest.com/scholarly-journals/integrating-chatgpt-into-software-development/docview/3181904918/se-2?accountid=11643>. Place: Wollongong

97. Takerngsaksiri, W., Warusavitarn, C., Yaacoub, C., Keng Hou, M.H., Tantithamthavorn, C.: Students' Perspectives on AI Code Completion: Benefits and Challenges. In: 2024 IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC), pp. 1606–1611. IEEE, Osaka, Japan (2024). DOI 10.1109/COMPSAC61105.2024.00252. URL <https://ieeexplore.ieee.org/document/10633447/>
98. Tang, N., Chen, M., Ning, Z., Bansal, A., Huang, Y., McMillan, C., Li, T.J.J.: Developer Behaviors in Validating and Repairing LLM-Generated Code Using IDE and Eye Tracking. In: 2024 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC), pp. 40–46 (2024). DOI 10.1109/VL/HCC60511.2024.00015. Journal Abbreviation: 2024 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)
99. Tian, Y., Yan, W., Yang, Q., Zhao, X., Chen, Q., Wang, W., Luo, Z., Ma, L., Song, D.: CodeHalu: Investigating Code Hallucinations in LLMs via Execution-based Verification. In: Proceedings of the AAAI Conference on Artificial Intelligence (AAAI-25), vol. 39, pp. 25300–25308. Association for the Advancement of Artificial Intelligence (AAAI) (2025). DOI 10.1609/aaai.v39i24.34717. URL <https://ojs.aaai.org/index.php/AAAI/article/view/34717>
100. Tony, C., Díaz Ferreyra, N.E., Mutas, M., Dhif, S., Scandariato, R.: Prompting Techniques for Secure Code Generation: A Systematic Investigation. *ACM Transactions on Software Engineering and Methodology* **34**(8), 1–53 (2025). DOI 10.1145/3722108. URL <https://dl.acm.org/doi/10.1145/3722108>
101. Tufano, R., Pepe, F., Zampetti, F., Mastropaolo, A., Dabić, O., Di Penta, M., Bavota, G.: Developers and Generative AI: A Study of Self-Admitted Usage in Open Source Projects. *Empirical Software Engineering* **31**(4), 108 (2026). DOI 10.1007/s10664-026-10848-w. URL <https://www.proquest.com/scholarly-journals/developers-generative-ai-study-self-admitted/docview/3324332841/se-2?accountid=11643>. Place: Dordrecht
102. Twist, L., Zhang, J.M., Harman, M., Yannakoudakis, H.: Library Hallucinations in LLMs: Risk Analysis Grounded in Developer Queries (2025). DOI 10.48550/ARXIV.2509.22202. URL <https://arxiv.org/abs/2509.22202>. Version Number: 2
103. Vaillant, T.S., Almeida, F.D.d., Neto, P.A.M.S., Gao, C., Bosch, J., Almeida, E.S.d.: Developers' Perceptions on the Impact of ChatGPT in Software Development: A Survey (2024). DOI 10.48550/arXiv.2405.12195. URL <http://arxiv.org/abs/2405.12195>. ArXiv:2405.12195 [cs]
104. Vangala, B.P., Adibifar, A., Gehani, A., Malik, T.: AI-Generated Code Is Not Reproducible (Yet): An Empirical Study of Dependency Gaps in LLM-Based Coding Agents (2026). DOI 10.48550/arXiv.2512.22387. URL <http://arxiv.org/abs/2512.22387>. ArXiv:2512.22387 [cs]
105. Wang, R., Cheng, R., Ford, D., Zimmermann, T.: Investigating and Designing for Trust in AI-powered Code Generation Tools. In: The 2024 ACM Conference on Fairness, Accountability, and Transparency, pp. 1475–1493. ACM, Rio de Janeiro Brazil (2024). DOI 10.1145/3630106.3658984. URL <https://dl.acm.org/doi/10.1145/3630106.3658984>
106. Wang, W., Ning, H., Zhang, G., Liu, L., Wang, Y.: Rocks Coding, Not Development: A Human-Centric, Experimental Evaluation of LLM-Supported SE Tasks. *Proceedings of the ACM on Software Engineering* **1**(FSE), 699–721 (2024). DOI 10.1145/3643758. URL <https://dl.acm.org/doi/10.1145/3643758>
107. Waseem, M., Das, T., Ahmad, A., Liang, P., Fehmideh, M., Mikkonen, T.: ChatGPT as a Software Development Bot: A Project-based Study (2024). DOI 10.48550/arXiv.2310.13648. URL <http://arxiv.org/abs/2310.13648>. ArXiv:2310.13648 [cs.SE]
108. Weisz, J.D., Kumar, S.V., Muller, M., Browne, K.E., Goldberg, A., Heintze, K.E., Bajpai, S.: Examining the Use and Impact of an AI Code Assistant on Developer Productivity and Experience in the Enterprise. In: Proceedings of the Extended Abstracts of the CHI Conference on Human Factors in Computing Systems, pp. 1–13. ACM, Yokohama Japan (2025). DOI 10.1145/3706599.3706670. URL <https://dl.acm.org/doi/10.1145/3706599.3706670>
109. Xia, C.S., Deng, Y., Dunn, S., Zhang, L.: Agentless: Demystifying LLM-based Software Engineering Agents (2024). DOI 10.48550/arxiv.2407.01489. URL <https://arxiv.org/abs/2407.01489>
110. Xu, F., Medappa, P.K., Tunc, M.M., Vroegindewij, M., Fransoo, J.C.: AI-Assisted Programming Decreases the Productivity of Experienced Developers by Increasing the Technical Debt and Maintenance Burden (2026). DOI 10.48550/arXiv.2510.10165. URL <http://arxiv.org/abs/2510.10165>. ArXiv:2510.10165 [econ]

111. Xu, F.F., Vasilescu, B., Neubig, G.: In-IDE Code Generation from Natural Language: Promise and Challenges. *ACM Transactions on Software Engineering and Methodology* **31**(2), 1–47 (2022). DOI 10.1145/3487569. URL <https://dl.acm.org/doi/10.1145/3487569>
112. Yan, D., Gao, Z., Liu, Z.: A Closer Look at Different Difficulty Levels Code Generation Abilities of ChatGPT. In: 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 1887–1898. IEEE, Luxembourg, Luxembourg (2023). DOI 10.1109/ASE56229.2023.00096. URL <https://ieeexplore.ieee.org/document/10298505/>
113. Yin, Z., Gao, C., Fan, C., Yang, W., Xue, Y., Zhang, L.: A Comprehensive Empirical Evaluation of Agent Frameworks on Code-centric Software Engineering Tasks (2025). DOI 10.48550/arXiv.2511.00872. URL <http://arxiv.org/abs/2511.00872>. ArXiv:2511.00872 [cs]
114. Zhang, X., Jiang, W., Zhai, J., Ma, S., Bao, Q., Lin, C., Shen, C., Li, T., Liu, Y.: Rethinking Technology Stack Selection with AI Coding Proficiency (2025). DOI 10.48550/arXiv.2509.11132. URL <http://arxiv.org/abs/2509.11132>. ArXiv:2509.11132 [cs.SE]
115. Zheng, D., Wang, Y., Shi, E., Liu, X., Ma, Y., Zhang, H., Zheng, Z.: Top General Performance = Top Domain Performance? DomainCodeBench: A Multi-domain Code Generation Benchmark (2025). DOI 10.48550/arXiv.2412.18573. URL <http://arxiv.org/abs/2412.18573>. ArXiv:2412.18573 [cs.SE]
116. Zhong, S., Zou, Y., Adams, B.: Developer-LLM Conversations: An Empirical Study of Interactions and Generated Code Quality (2025). DOI 10.48550/arXiv.2509.10402. URL <http://arxiv.org/abs/2509.10402>. ArXiv:2509.10402 [cs]
117. Zhong, Z., Raghunathan, A., Carlini, N.: ImpossibleBench: Measuring LLMs' Propensity of Exploiting Test Cases (2025). DOI 10.48550/arXiv.2510.20270. URL <http://arxiv.org/abs/2510.20270>. ArXiv:2510.20270 [cs.LG]